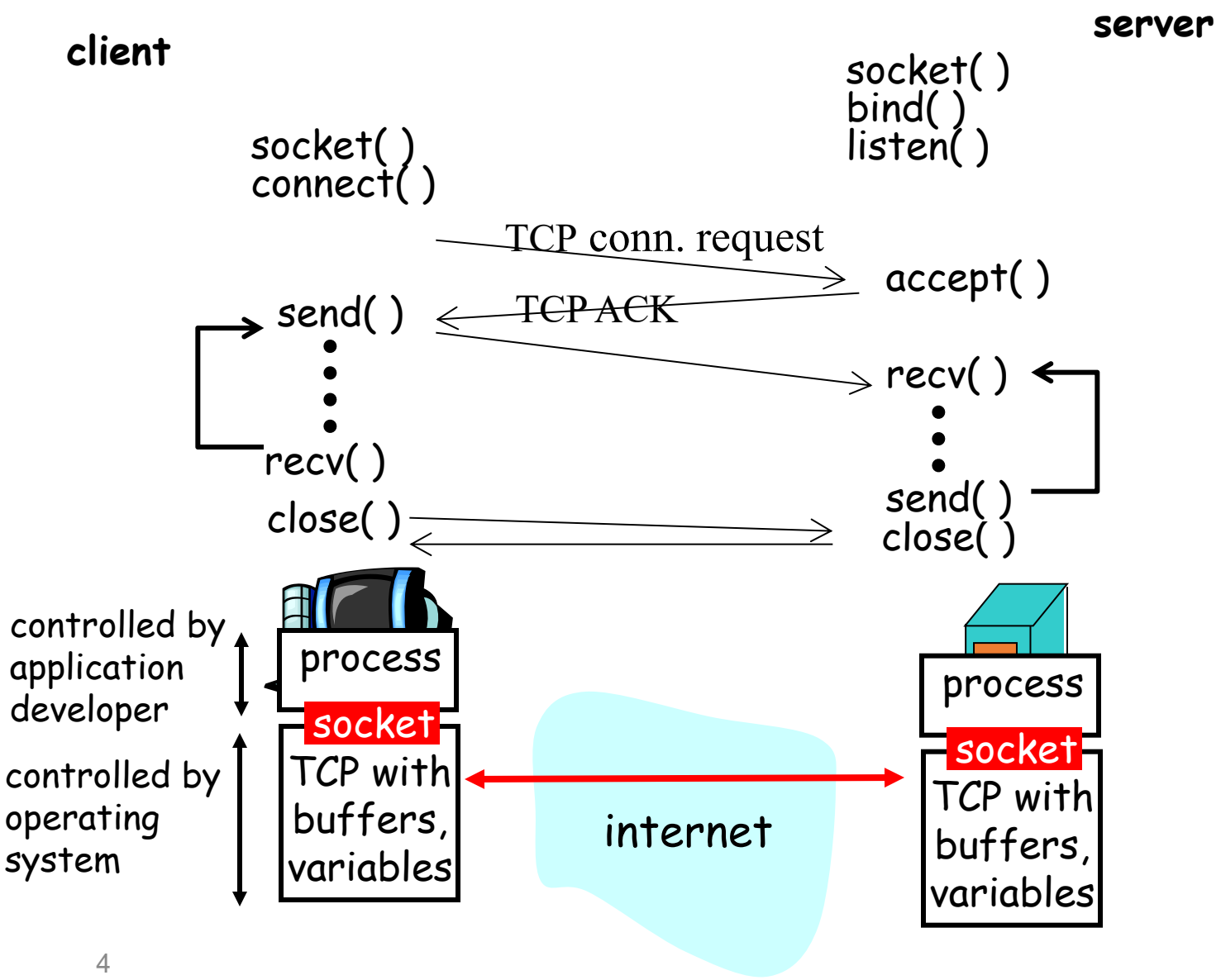
# *An Overview Of*
# *The TCP's Sockets API*

השקפים המסומנים ב- מהווים חומר הרחבה.

התוכן המובא בשקפים אלו <u>אינו</u> מהווה חלק מהחומר המחייב לבחינה בקורס.

# *TCP client & server: overall interaction*

# Socket-programming using TCP




- השקף מתאר את האינטראקציה (הכוללת !) בין תכנית לקוח, cli, לבית תכנית שרת, srv.
- התכניות מחליפות ביניהן מידע [= הודעות] באמצעות קישור TCP
היינו: באמצעות פרוטוקול תקשורת שכבה 5: P

תיאור מדויק יותר מהנ"ל: צד הלקוח של הפרוטוקול, Pcli וצד השרת של הפרוטוקול, Psrv, מחליפים הודעות באמצעות קישור TCP

- האינטראקציה נעשית ע"י כך ש- Pcli ו- Psrv קוראות לפונקציות [= מבצעות קריאות מערכת]
- לדוגמא: socket(), bind(), ...
- פונקציות אלו מהוות חלק מממשק הסוקטים
  - שתואר מספר פעמים במהלך ההרצאות עד כה.
  - בין היתר, בהרצאה שעסקה בתיאור מחזור החיים של תכנון המודל הרשתי TCP/IP

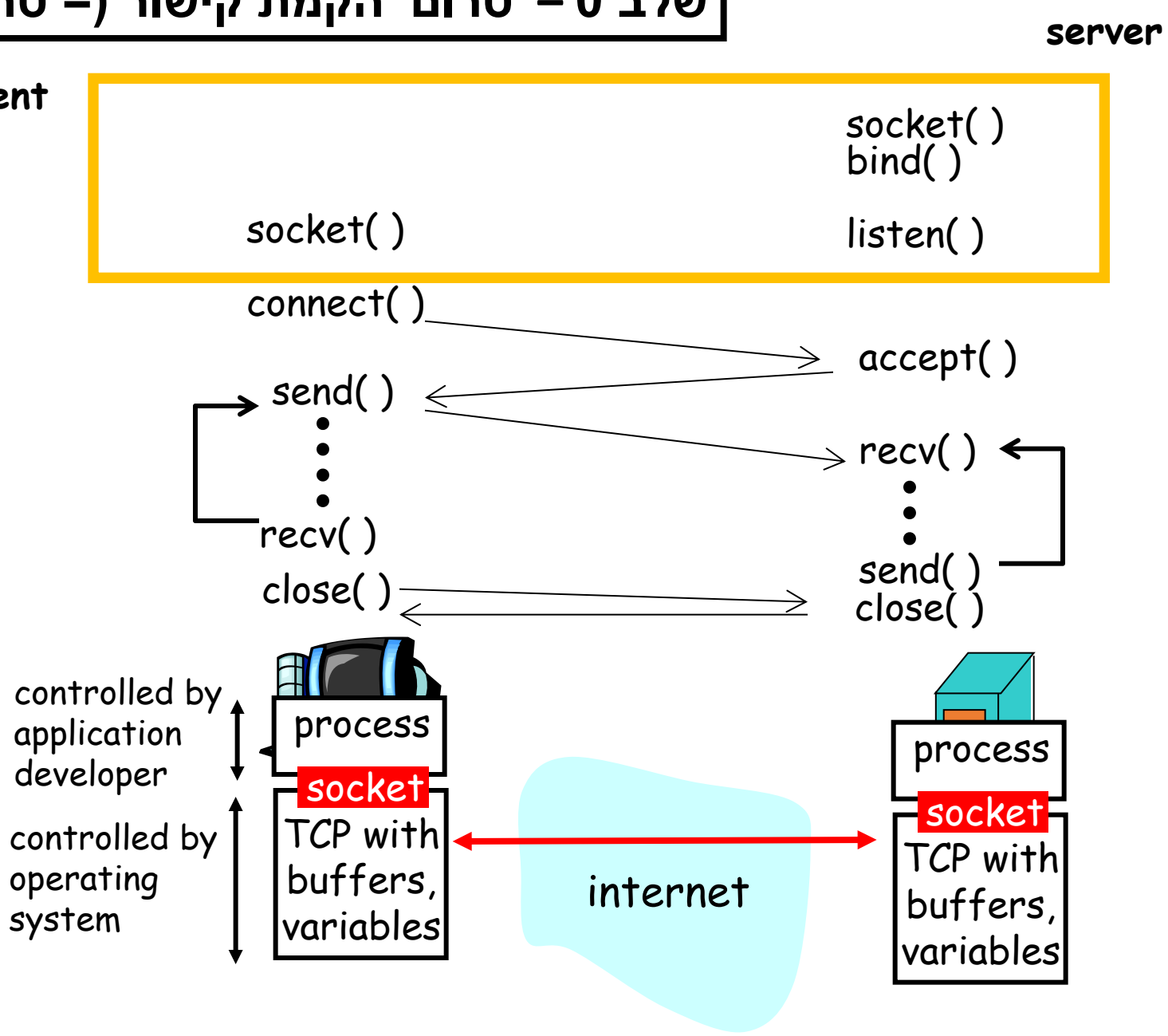
- המימוש של כל הפונקציות האלו ( socket, bind, listen, ...) נעשה ע"י מודול ה- TCP
- שהוא, כזכור, חלק ממודול ה- TCP
- היינו, הפונקציות האלו הן חלק ממאות (רבות) של פונקציות ש'ביחד' מהוות את TCP
[שהוא 'חלק' מעשרות ומאות אלפי פונקציות שביחד מהוות את מערכת ההפעלה]

# האינטראקציה הכוללת בין לקוח TCP ושרת TCP

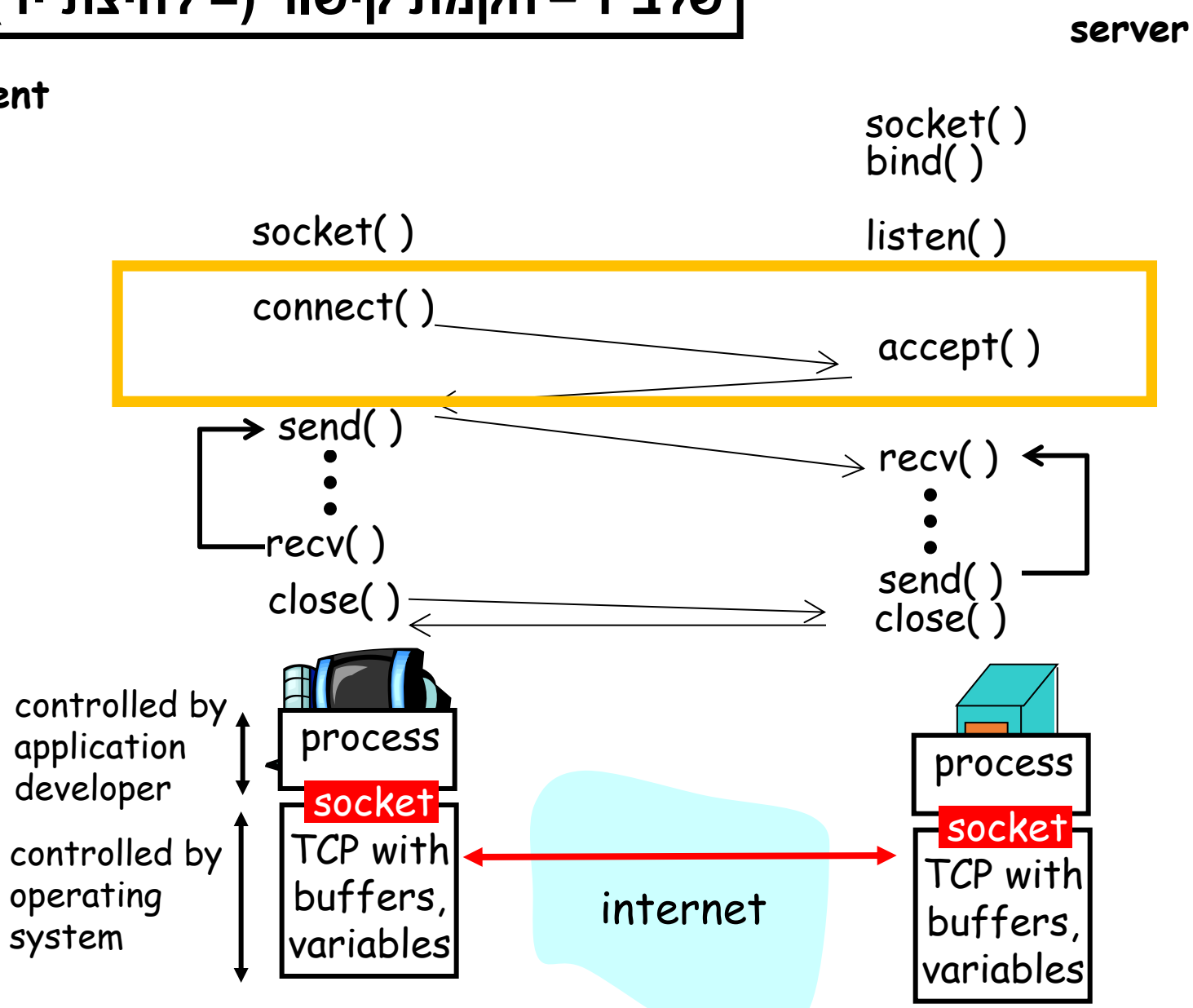
האינטראקציה הכוללת בין לקוח ושרת המחליפים ביניהם הודעות 'באמצעות' קישור TCP כוללת 4 שלבים:

- 0. 'טרום' לחיצת יד [= 'טרום' הקמת קישור]
- 1. הקמת קישור [= לחיצת יד תלת-שלבית]
- 2. החלפת דאטה בין הלקוח והשרת [= החלפת הודעות בקשה ותשובה]
  - סגמנטים ש'מכילים' הודעות או 'חלקי' הודעות
- 3. סגירת הקישור.

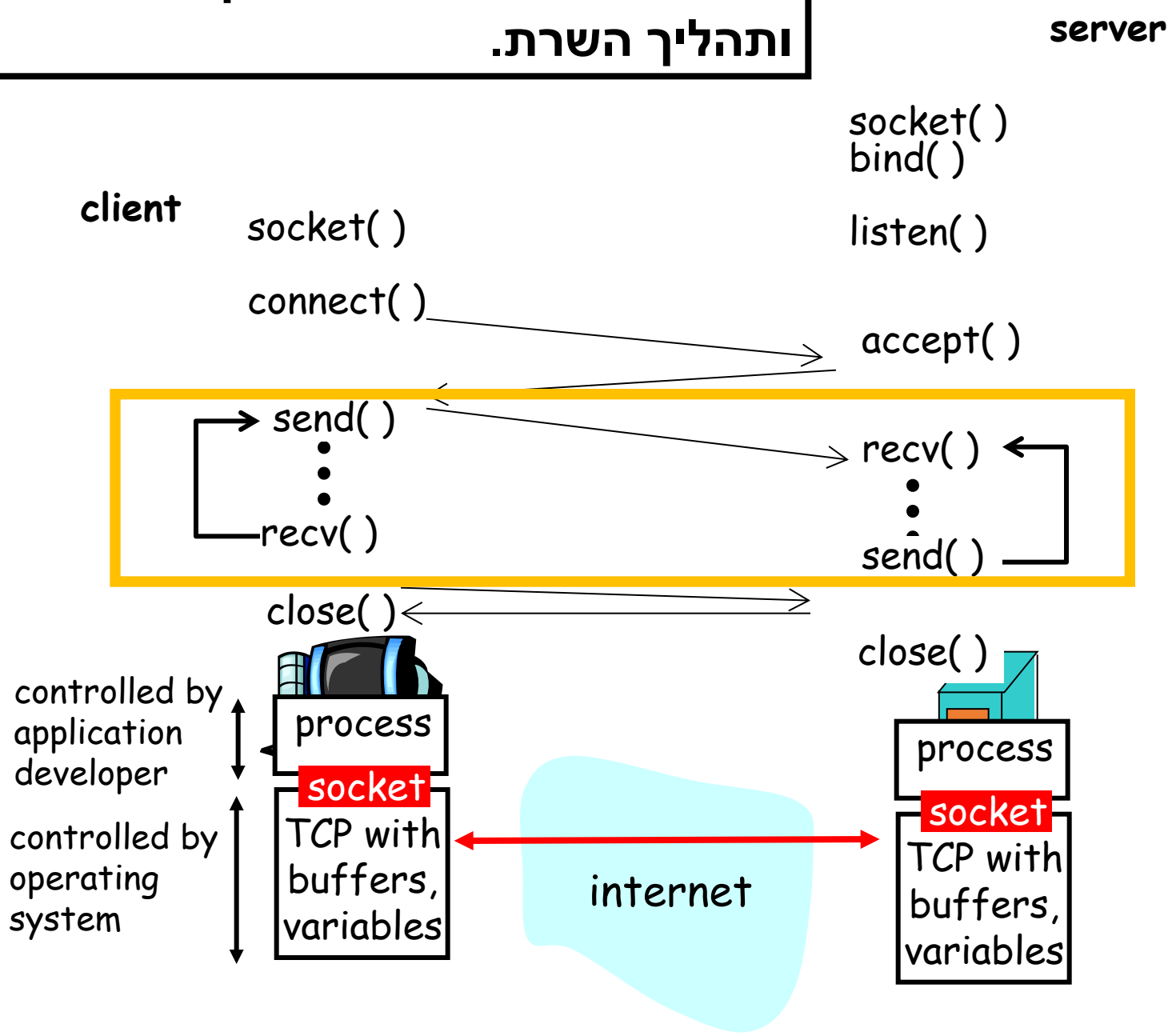
# שלב 0 – 'טרום' הקמת קישור (= טרום לחיצת יד)



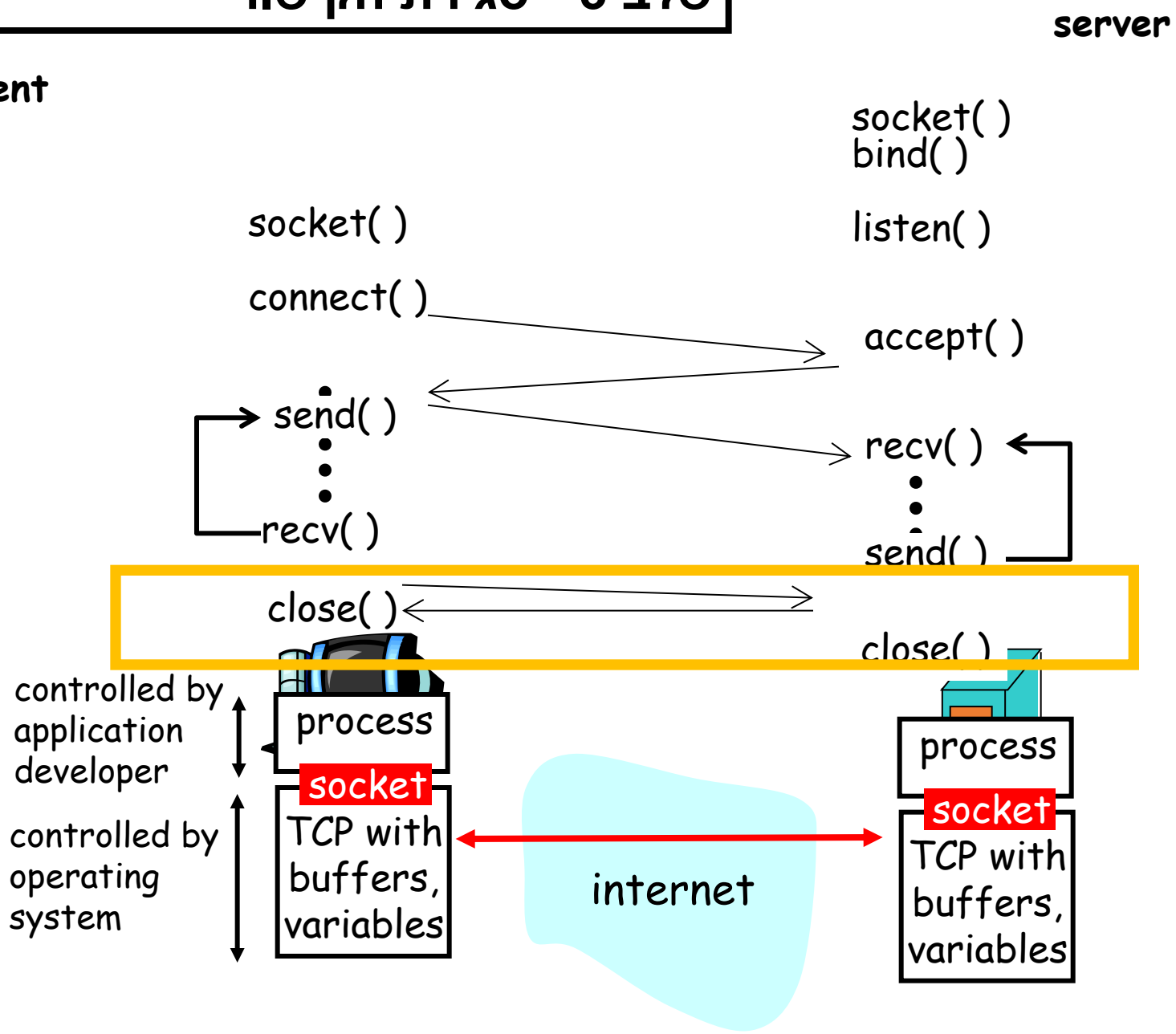
# שלב 1 – הקמת קישור (= לחיצת יד)



# שלב 2 – החלפת הודעות בקשה ותשובה בין תהליך הלקוח ותהליך השרת.



# שלב 3 – סגירת הקישור



*TCP's connection's Life Cycle*

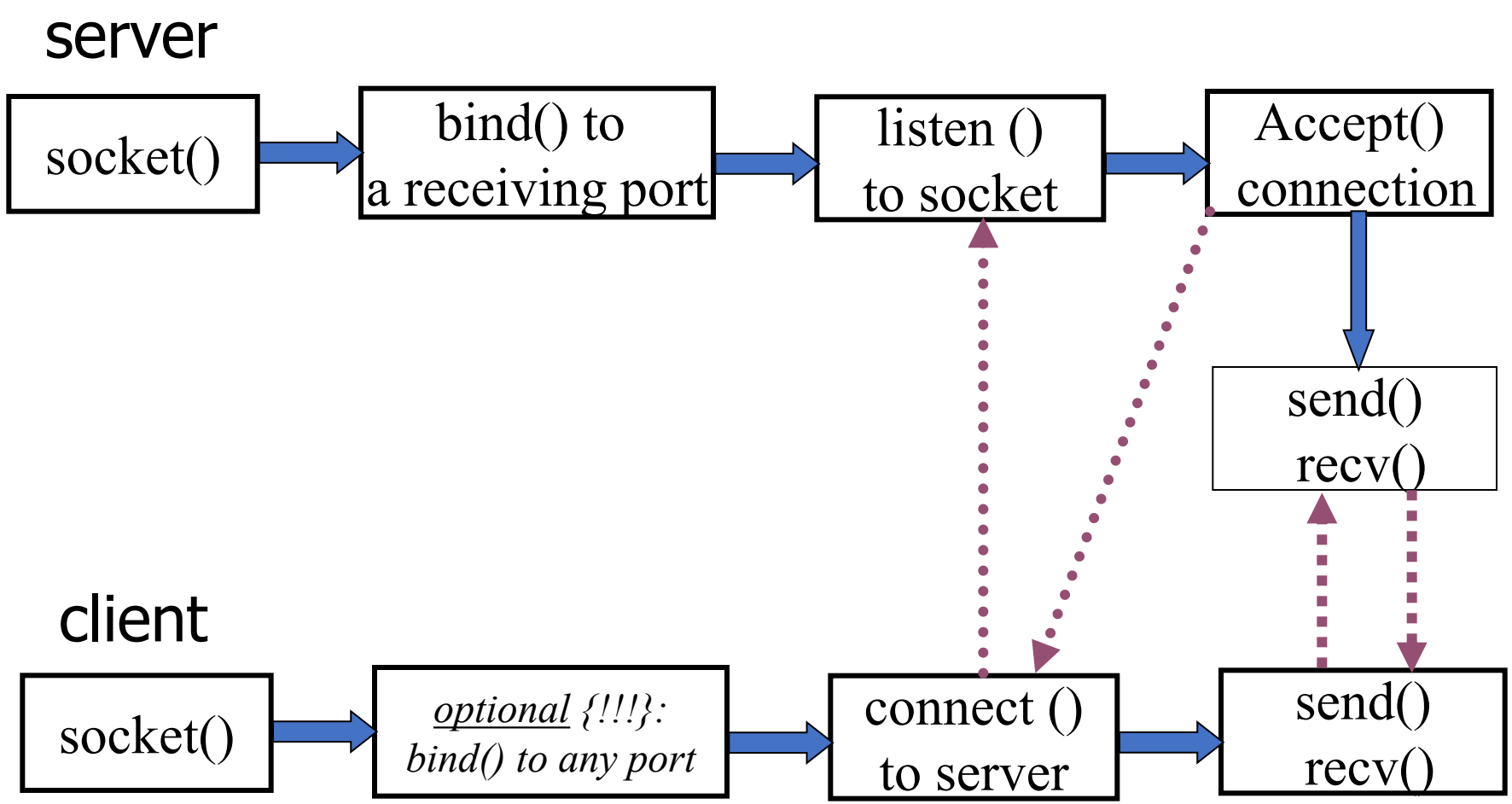
*vs.*

*UDP's connection's Life Cycle*

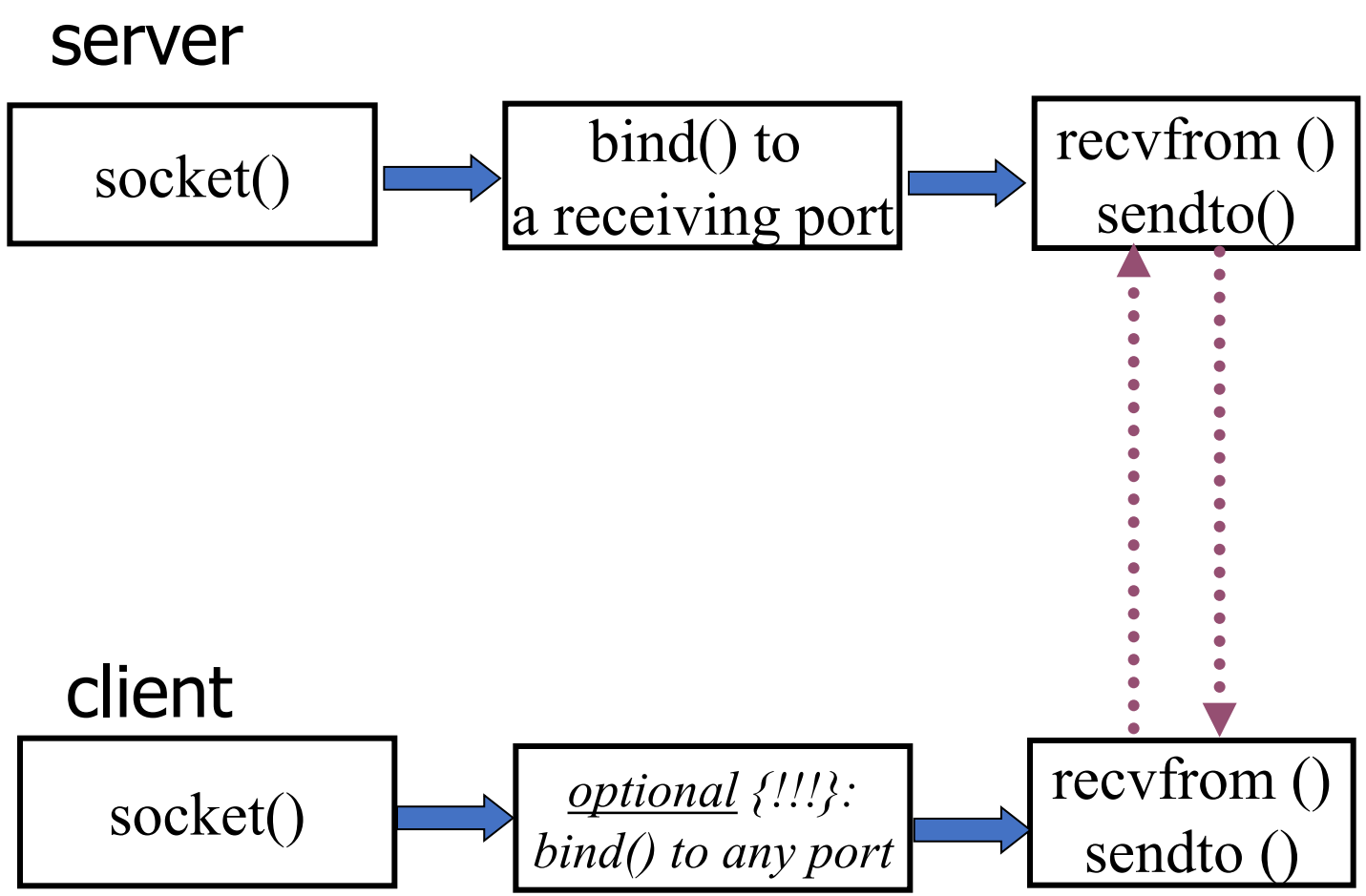
# קישורי TCP לעומת קישורי UDP

*למרות שהדיון הנוכחי שלנו עוסק ב- TCP, לצורך השוואה ראשונית של TCP ו- UDP נביא להלן:*
*- את סדרת הצעדים הדרושה להקמת קישור TCP ולהחלפת הודעות 'באמצעותו'*
*- לעומת סדרת הצעדים הדרושה להקמת קישור UDP ולהחלפת הודעות 'באמצעותו'*

# TCP - Client-Server communication via a TCP connection



# UDP - Client-Server communication via a UDP connection



# שלב 0: 'טרום' לחיצת יד

# מה מתבצע בשלב 'טרום הקמת קישור' ?

- בשלב הזה מתבצעים, באופן בלתי תלוי, צעדים הן בצד הלקוח והן בצד השרת, אשר חייבים להתבצע לפני שלב הניסיון להקמת קישור בין 2 הצדדים.

- בצד השרת:

א. בקשה להקצאת סוקט [קריאה ל- ()socket].
- הבקשה מופנית ל- TCP שבמחשב השרת, ומטופלת על ידו.

ב. 'קיבוע' ה- local port number של הסוקט שהוקצה [קריאה ל- ()bind]
- בהתאם לאפיון במסמך האפיון של הפרוטוקול

ג. 'העברת' הסוקט למצב האזנה [קריאה ל- ()listen]
- המתנה ל"קבלת' בקשות connect מתהליכי לקוח

- בצד הלקוח:

- צעד אחד ויחיד: בקשה להקצאת סוקט [קריאה ל- ()socket].
- הבקשה מופנית ל- TCP שבמחשב הלקוח, ומטופלת על ידו.

*socket()*

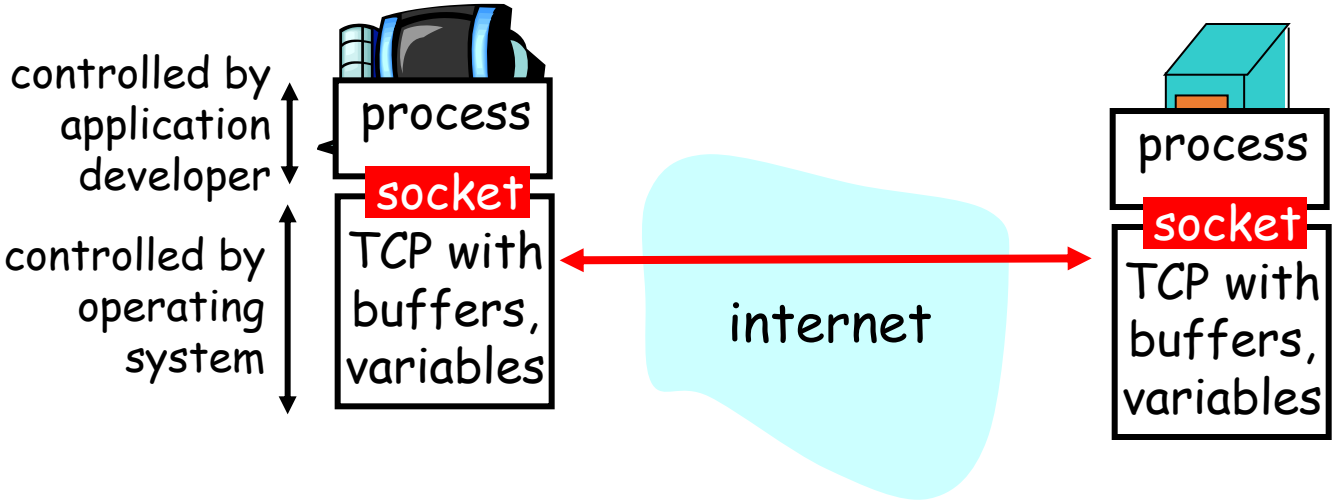
client program

running in a client computer C

a server program

running in a server computer S

socket( )



## בקשה להקצאת סוקט:
## קריאה ל- socket()

family: address family (protocol family)

AF_UNIX, AF_INET, AF_NS, AF_IMPLINK

AF_INET: associates a socket with the Internet protocol family

```
#include <sys/types.h>
#include <sys/socket.h>


int sd;        /* socket descriptor */
if ((sd = socket(AF_INET, SOCK_STREAM, 0)) < 0) }
        perror("socket");
        exit(1);
}
```

sock-type

SOCK_STREAM: selects the TCP protocol

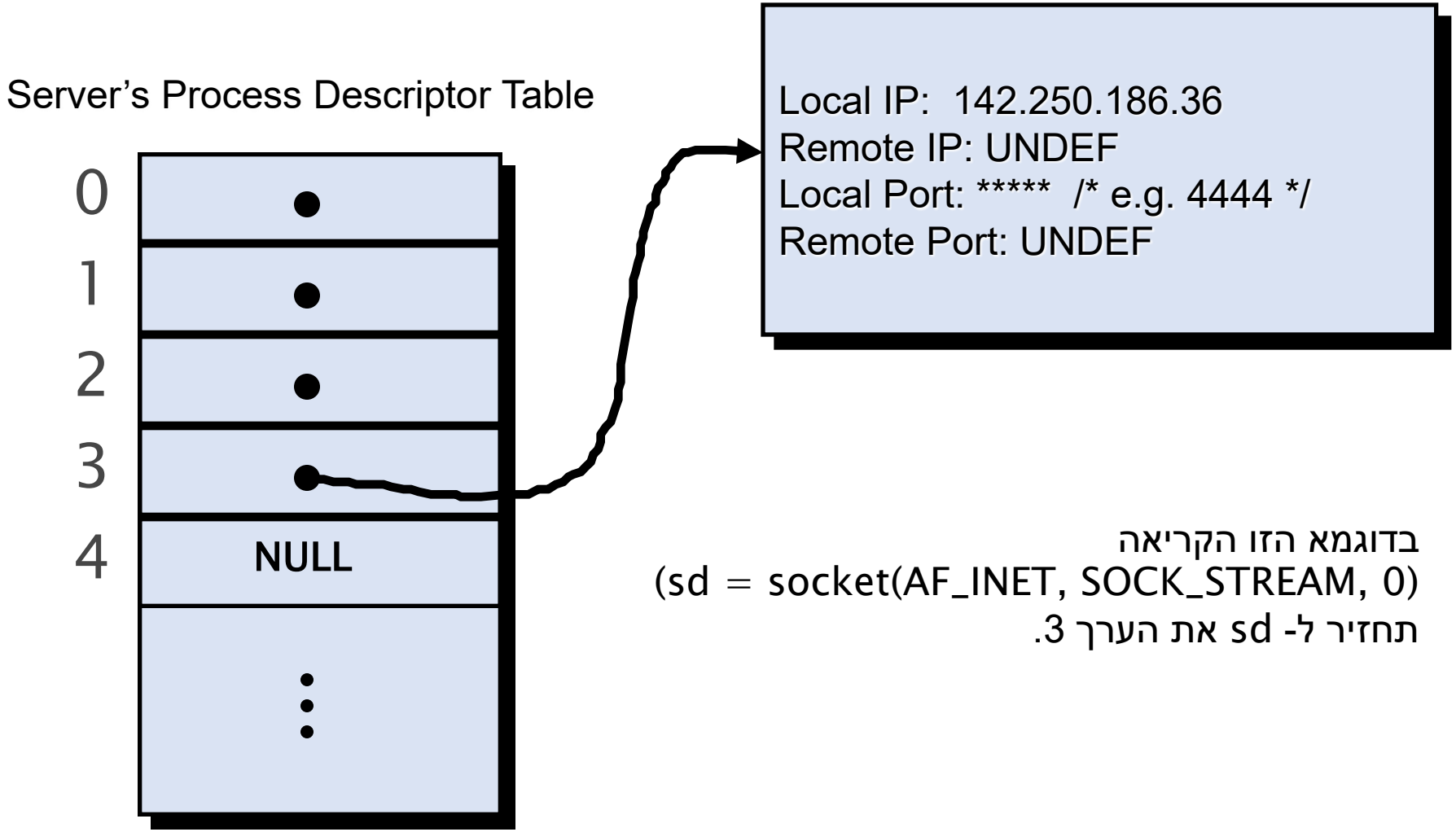
SOCK_DGRAM: selects the UDP protocol

return value: **socket descriptor** (of type int)

[socket descriptors are similar to file descriptors]

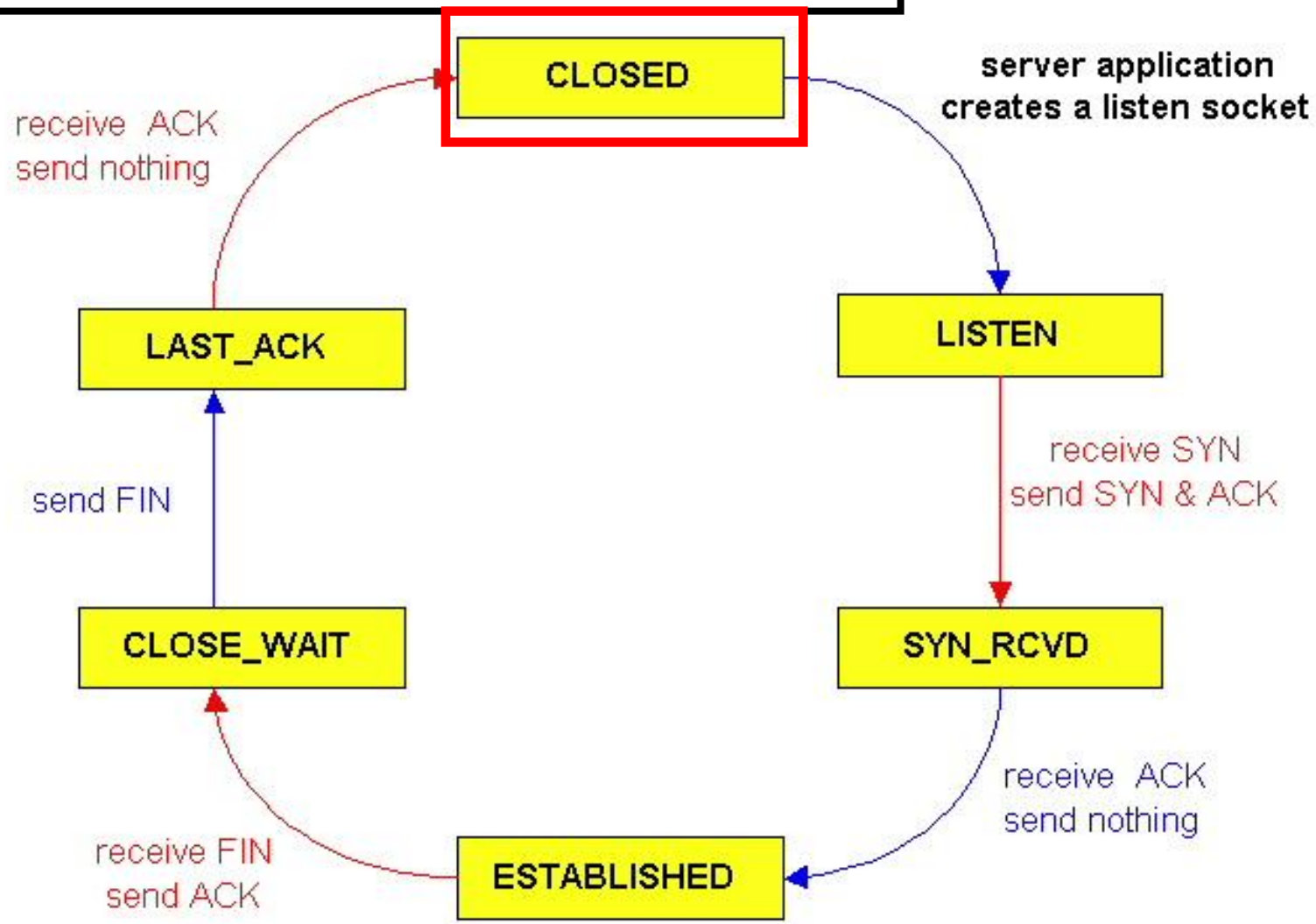
'-1' indicates that an error occurred

# The socket struct




בדוגמא הזו הקריאה
(sd = socket(AF_INET, SOCK_STREAM, 0)
תחזיר ל- sd את הערך 3.

**<u>Server's Socket Lifecycle:</u>**
**the server's socket state just after socket was allocated**



# *The socket struct structure*

# רשומת סוקט (טיפוסית)

- רשומת סוקט היא 'מקרה פרטי' של 'רשומת קובץ'
- שכן סוקט הוא סוג מסוים של קבצים

- רשומה כזו מוקצית בעקבות הקריאה ל- socket(...)
- היא מוקצית ע"י TCP או UDP {כתלות בסוג הסוקט]
- היא כוללת [= מורכבת מ] עשרות רבות של נתונים [= שדות] על הסוקט שדרושים לצורך הניהול ה'שוטף' של הסוקט {ע"י TCP או UDP]

# A typical socket struct

```
/* Connection details */
ushort type;
ushort Local_Port;
ushort Remote_Port;
struct netaddr Local_IP;
struct netaddr Remote_IP;

ushort state;
…
```

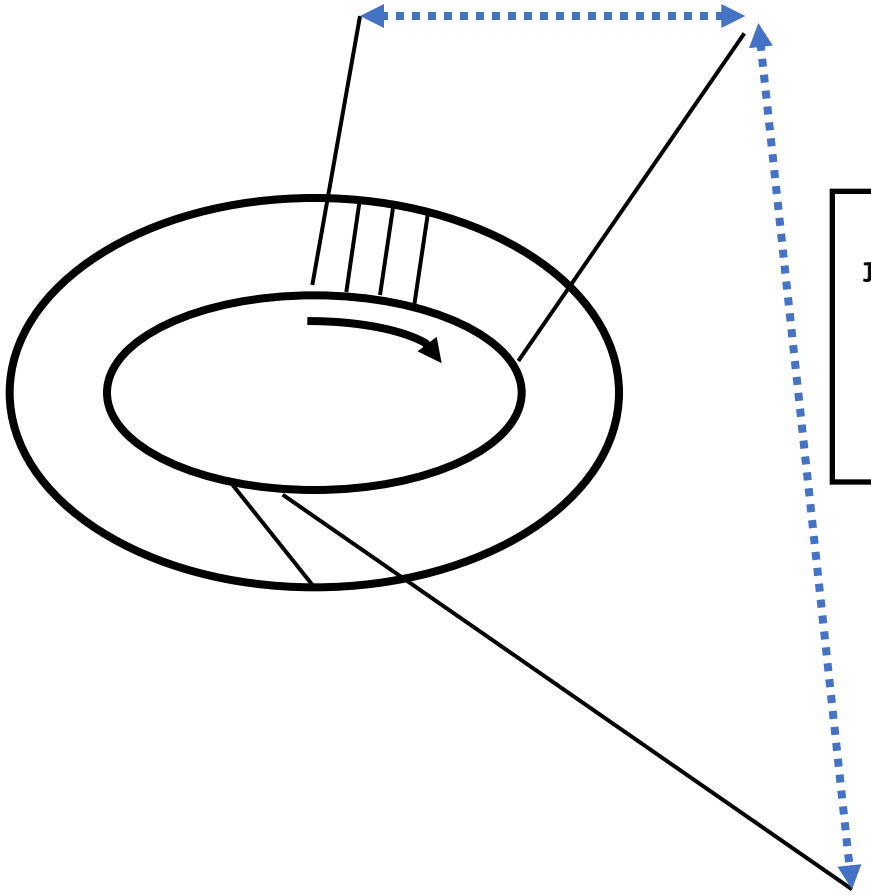
```
…
/* Send buffer */
uchar out[TCP_OBLEN];
uint ocount;

/* Send variables */
ushort sndmss;

 /* Initial Sequential Number. random value */
uint isn;

uint  seqno ; /* initial value = isn */
tcpseq snduna;
tcpseq sndnxt;

uint sndwnd;
uint sndcwn;
uint sndsst;

tcpseq sndup;
tcpseq sndwl1;
tcpseq sndwl2;
tcpseq iss;
tcpseq sndfin;
uchar sndflg;
…
```

```
…
int sndrtt;
int sndrtd;
int rxttime;
uint rxtcount;
int psttime;
```

```
…
/* Receive buffer */
uchar in[TCP_IBLEN];
uint ibytes;
uint istart;
uint icount;
uint inxt;

/* Receive variables */
tcpseq rcvnxt;
tcpseq rcvwnd;
tcpseq rcvup;
tcpseq rcvfin;
ushort rcvmss;
uchar rcvflg;
```

*buffers*

בתים ש
- הועתקו לחוצץ השליחה, ממרחב הכתובות של התהליך השולח { לאחר שהוא קרא ל- send() }
- *עדיין לא נשלחו*



מקום ריק
לתוכו ניתן להעתיק בתים נוספים {לאלו שכבר מצויים בחוצץ השליחה} ממרחב הכתובות של התהליך השולח,

כמובן במידה והתהליך השולח יבקש לשלוח בתים נוספים {ע"י קריאה/ות נוספת/ות ל- send()

בתים ש:
- הועתקו לחוצץ השליחה, ממרחב כתובות של התהליך השולח
- *נשלחו*
- *עדיין לא הגיע אישור (ACK) על קבלתם התקינה*

*implementation of the socket() call*

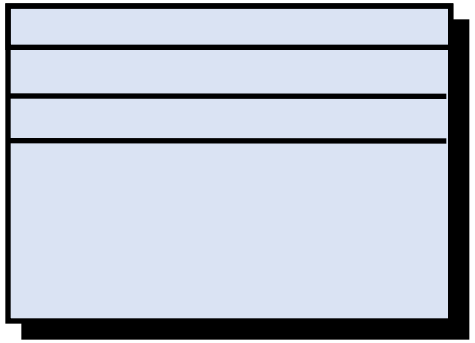
- הרוטינה socket מאתרת כניסה פנויה בטבלת הקבצים הפתוחים ברשומת ה- PCB של התהליך שביצע הקריאה ל- socket()
- כותבת בכניסה זו מצביע לרשומת הסוקט שיצרה
- מחזירה לתהליך את מספר האינדקס של כניסה זו בטבלת הקבצים הפתוחים.
  - אל מספר אינדקס זה אנו מתייחסים תחת השם socket descriptor
    - במקביל למונח file descriptor

# Process's PCB
# (Process Control Block)

process_id:
parent_process_id:

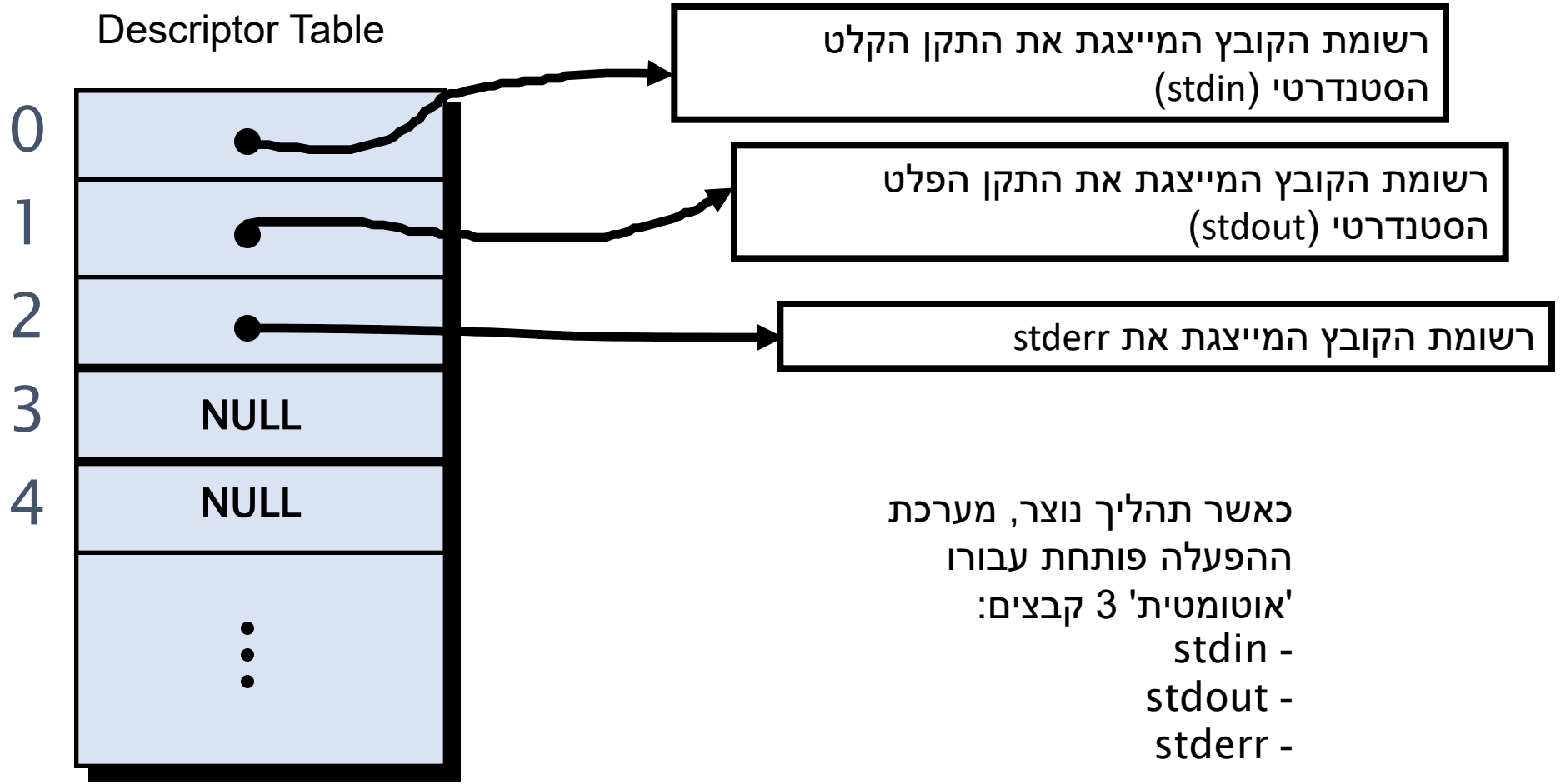
priority:
page table:

descriptor table:



= descriptor table
טבלת הקבצים הפתוחים של
התהליך.

# Descriptor table

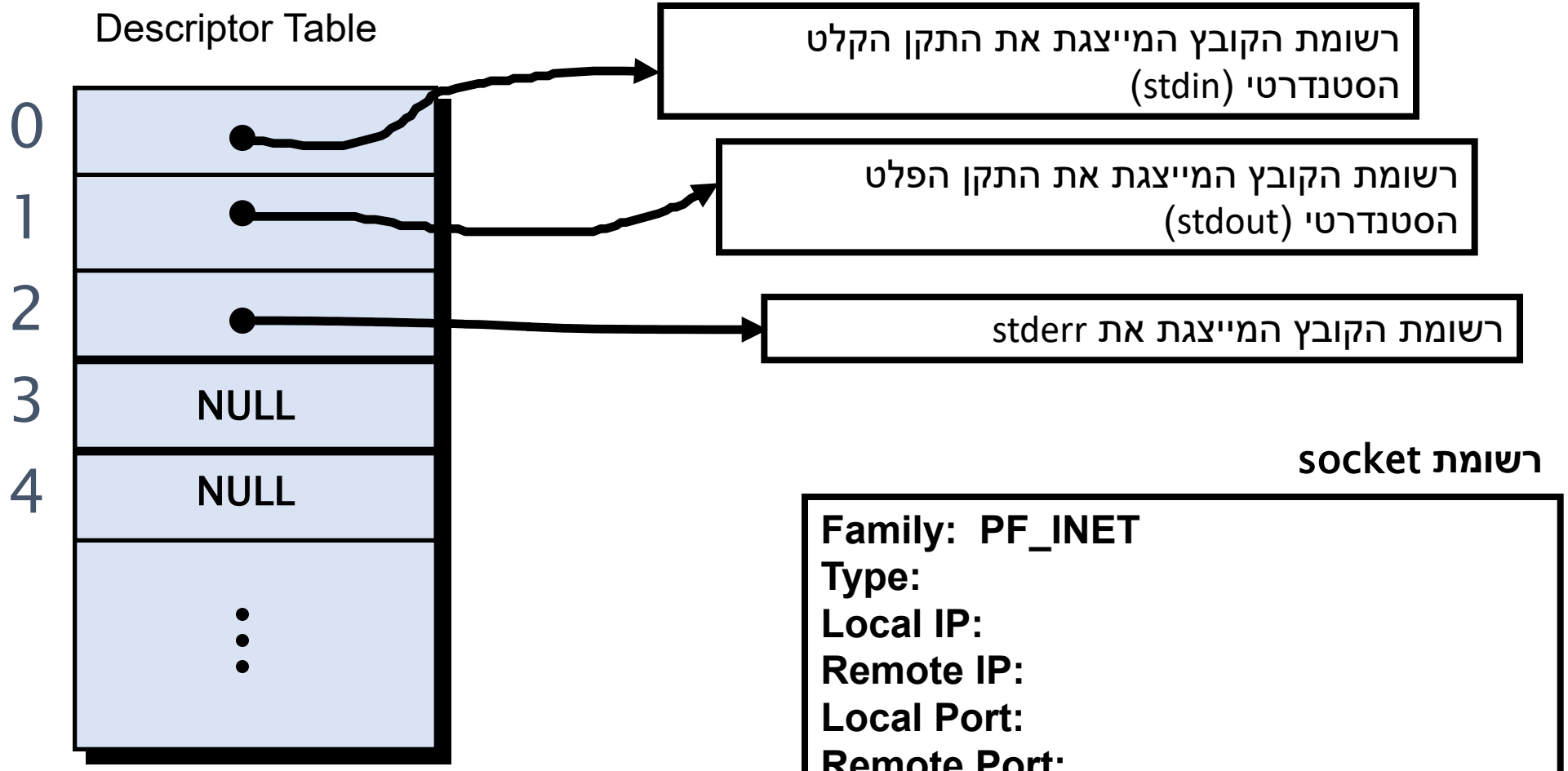



כאשר תהליך נוצר, מערכת ההפעלה פותחת עבורו 'אוטומטית' 3 קבצים:

- stdin
- stdout
- stderr

# Socket Descriptor Data Structure

הצעד הראשון בעקבות הקריאה
(sd = socket(AF_INET, SOCK_STREAM, 0)
הוא להקצות רשומת סוקט.

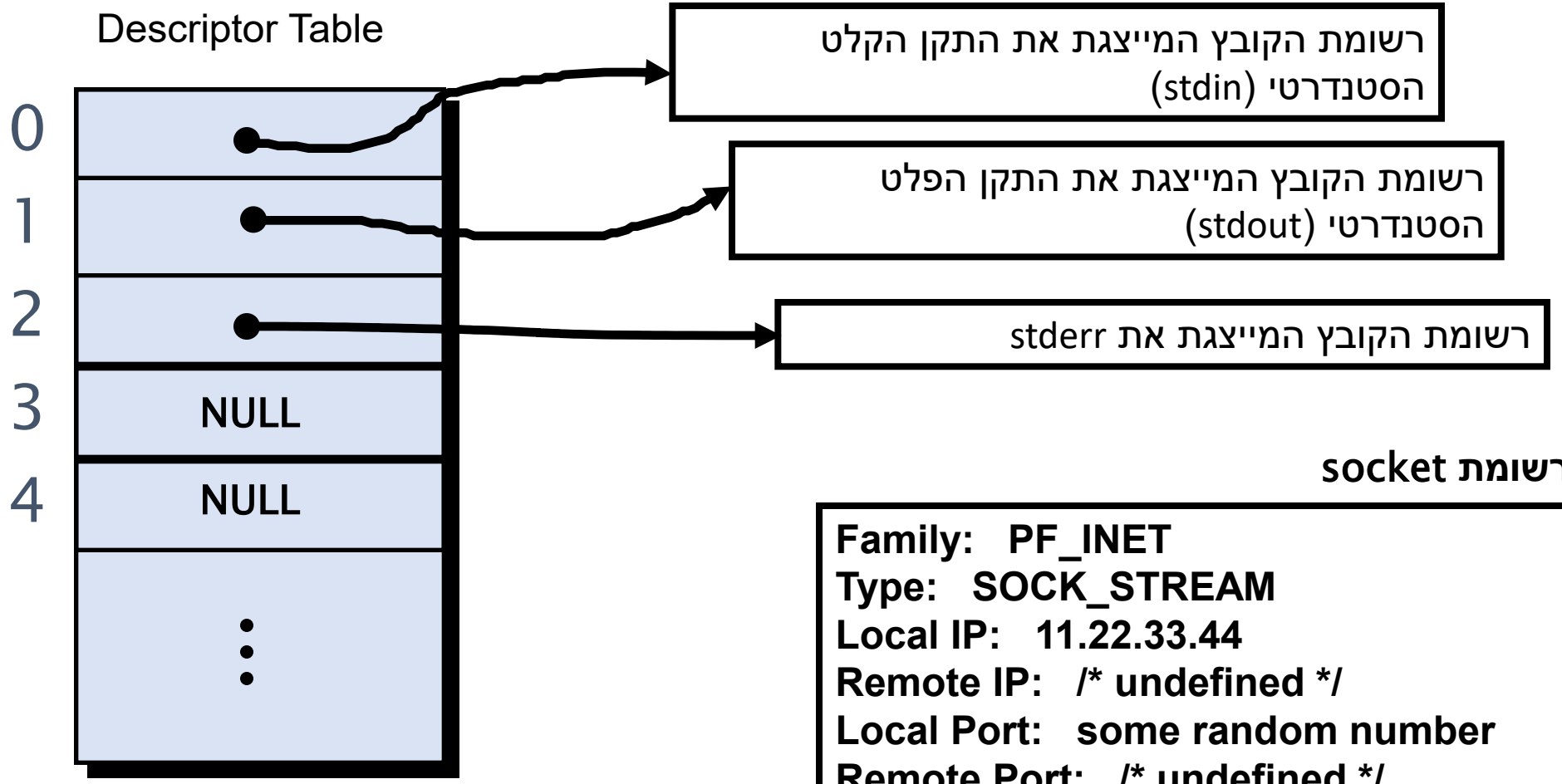



**רשומת socket**

**Family: PF_INET**
**Type:**
**Local IP:**
**Remote IP:**
**Local Port:**
**Remote Port:**

# Socket Descriptor Data Structure

הצעד השני בעקבות הקריאה
sd = socket(AF_INET, SOCK_STREAM, 0))
הוא לרשום נתונים בשדות של רשומת הסוקט.

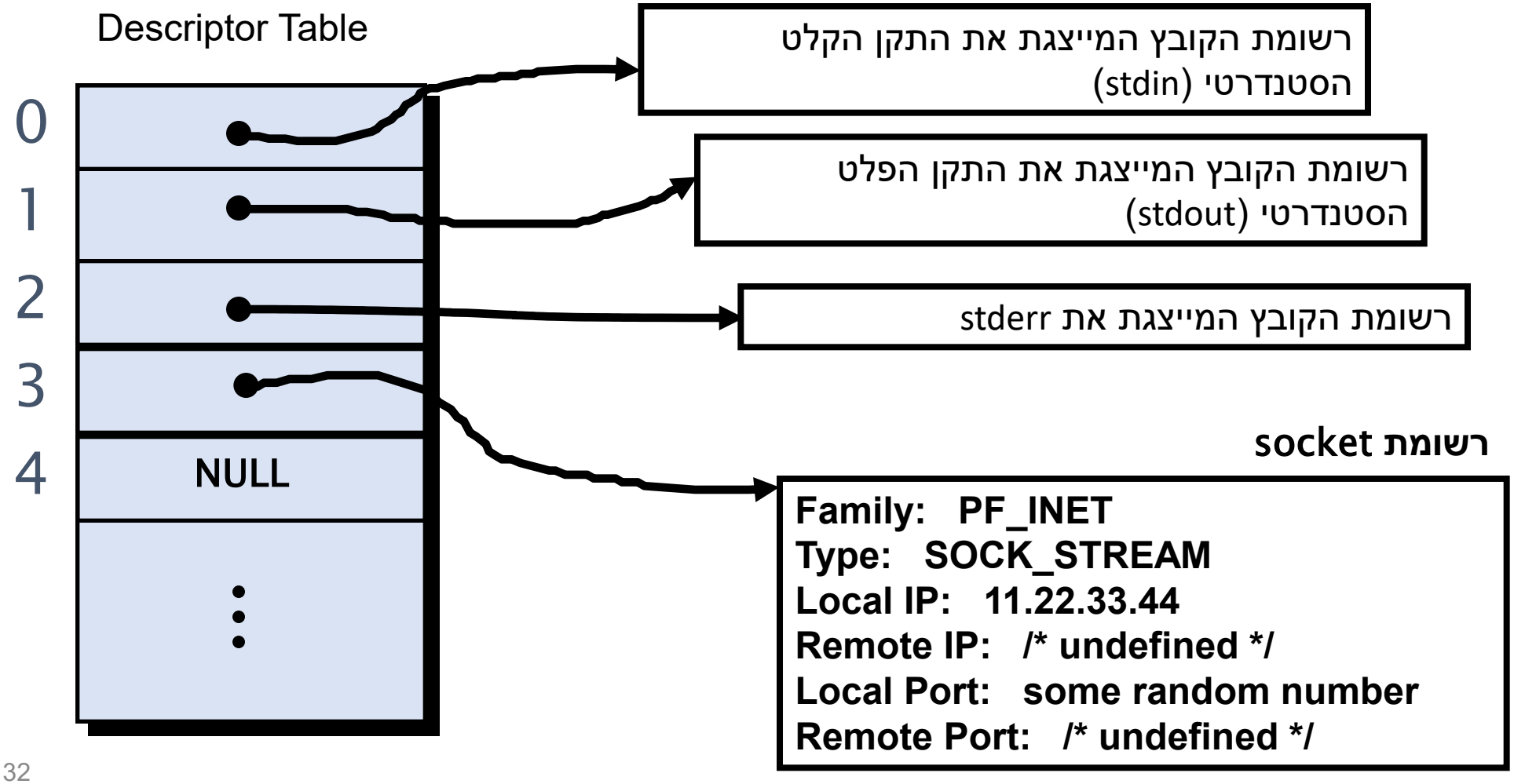



**רשומת socket**

**Family: PF_INET**
**Type: SOCK_STREAM**
**Local IP: 11.22.33.44**
**Remote IP: /* undefined */**
**Local Port: some random number**
**Remote Port: /* undefined */**

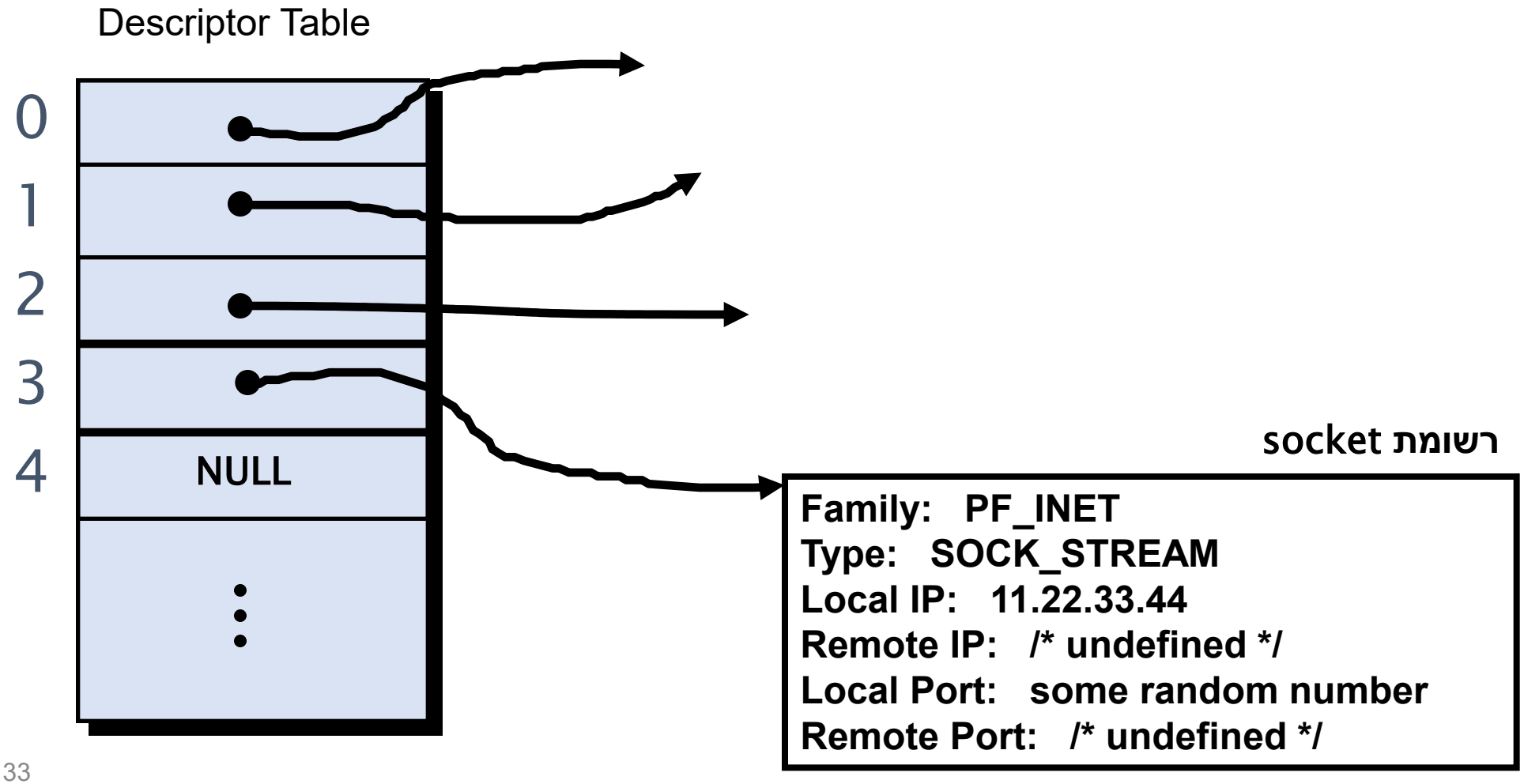
# Socket Descriptor Data Structure

הצעד השלישי בעקבות הקריאה
sd = socket(AF_INET, SOCK_STREAM, 0))
הוא לשמור מצביע לרשומת הסוקט.



# Socket Descriptor Data Structure

בדוגמא הזו הקריאה
sd = socket(AF_INET, SOCK_STREAM, 0))
תחזיר ל- sdאת הערך 3.



*server – bind()*
*and bind() implementation*

## *What is bind ?*

*- Once the server has a socket, it needs to associate (= bind) that socket with a <u>specific port number</u>*
*- A specific <u>local</u> [= on the server computer system] port number*

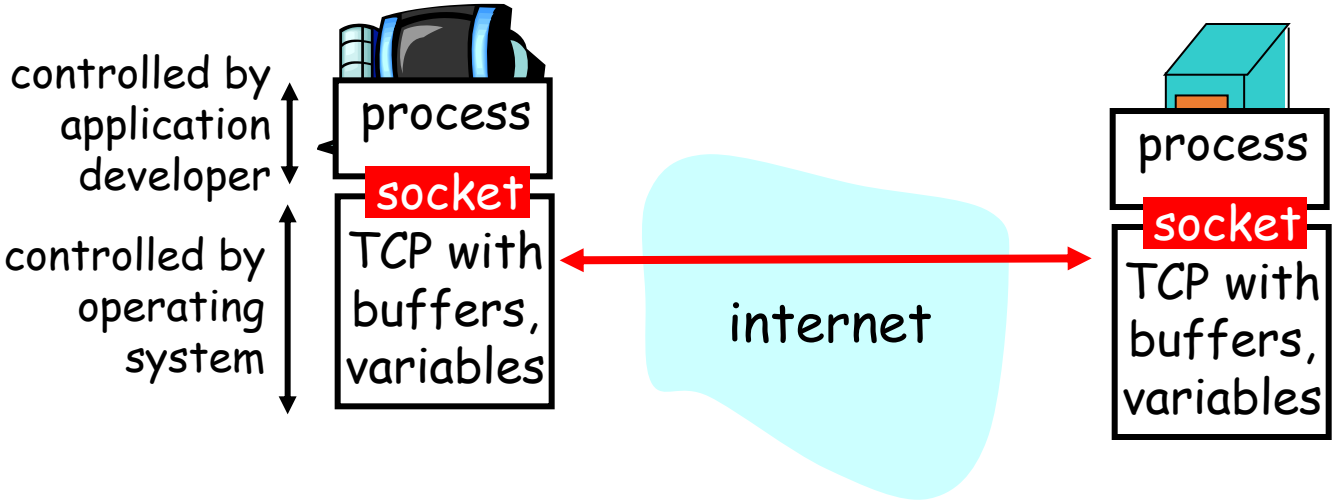
client program

running in a client computer C

a server program

running in a server computer S

socket( )
bind()



## Why the server's process <u>must</u> invoke bind() ?

- בקריאה ל- socket נוצרת רשומת סוקט, וערך השדה local port שלה הוא <u>ערך אקראי</u> בין 0 ל- 65,535

- כאמור הסוקט הזה ישמש את ה- P-server כסוקט האזנה.
- ולכן, מספר הפורט של סוקט זה <u>חייב להיות מספר הפורט שהוגדר במסמך האיפיון של P</u>
  - שכן אחרת P-client לא יוכל לבקש הקמת קישור TCP עם P-server (שכן, לא ידע למי להפנות בקשת הקמת הקישור)

## Why the server's process <u>must</u> invoke bind() ?

- לדוגמא, מסמך האפיון של פרוטוקול HTTP קובע כי מספר הפורט של http server הוא 80

- ולכן P-server לא 'יכול להישאר' עם ערך כלשהו (אקראי) בשדה local port (ערך שהקצה לו TCP במחשב S)
- והוא חייב 'לגרום לכך' (= לגרום ל- TCP) שהערך בשדה local port ברשומת הסוקט של סוקט ההאזנה יהיה 80 !
- הוא חייב לבצע שינוי זה <u>לפני</u> הקריאה ל- listen( ... ) !!!

## How the server's process invokes bind() ?

```
int sd;                         /* socket descriptor */
struct sockaddr_in srv;         /* used by bind() */


if ((sd = socket(AF_INET, SOCK_STREAM, 0)) < 0) }
            perror("socket");
            exit(1);
}

srv.sin_family = AF_INET;
srv.sin_addr.s_addr = htonl(INADDR_ANY);
srv.sin_port = htons(80);   /* host-to-network short */


if(bind(sd, (struct sockaddr*) &srv, sizeof(srv)) < 0) {
            perror("bind");
            exit(1);
}
```

*How bind is implemented ?*

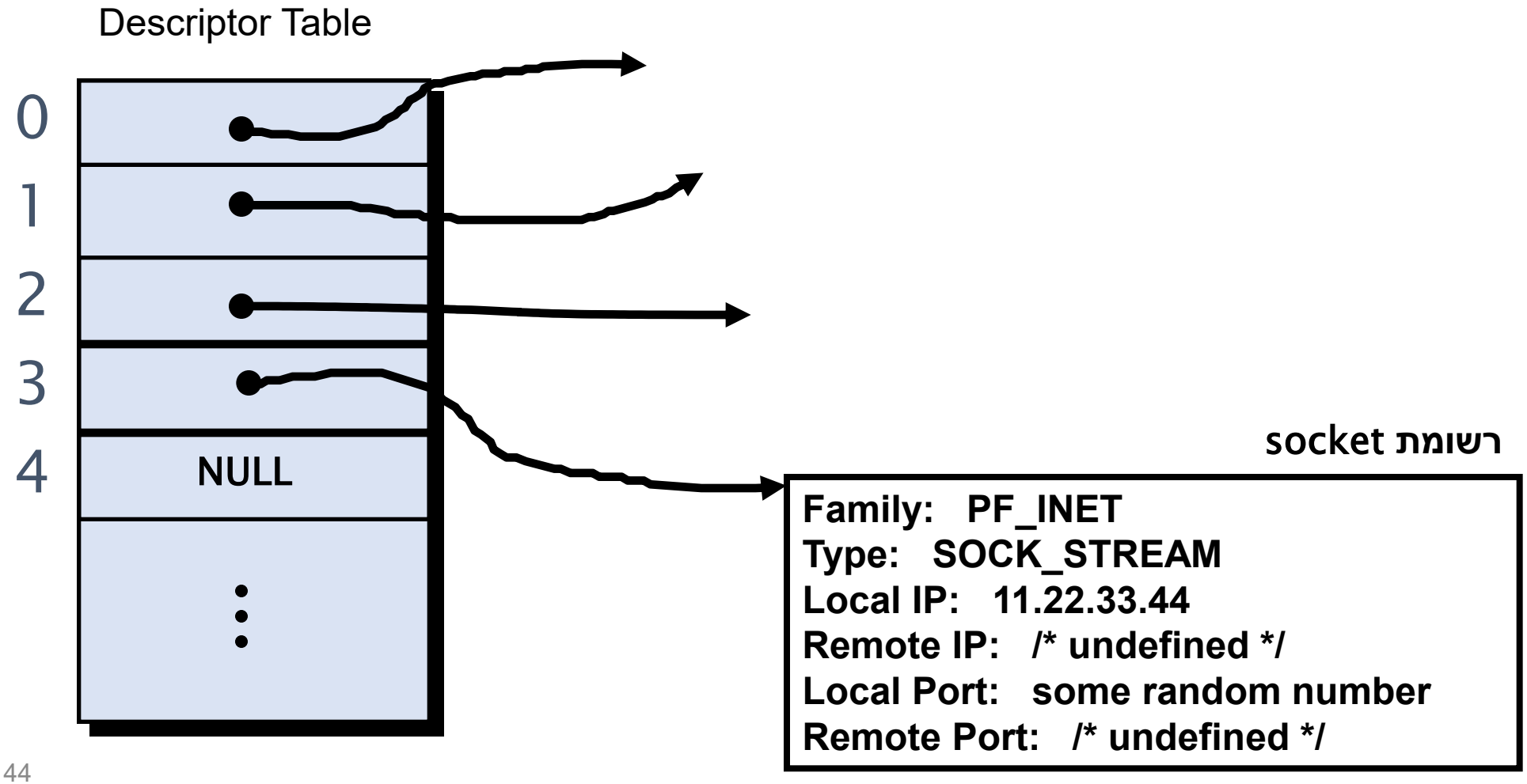
- הקריאה bind() היא קריאת מערכת
- הרוטינה bind עצמה ממומשת על ידי TCP
  - כחלק ממכלול הקוד [ = מאות רוטינות] שמממש את TCP
    - כחלק ממאות אלפי הרוטינות שמממשות את OS

- *bind:*
- על סמך הארגומנט הראשון של bind, ז"א על סמך הערך במשתנה sd, TCP ניגש לרשומת ה- PCB של התהליך שקרא ל- bind (ז"א, ל- PCB של תהליך ה- Apache)
- לטבלת הקבצים הפתוחים [= descriptor table] שברשומת ה- PCB
- אל כניסה מספר sd ב- descriptor table [ = הערך ששמור בארגומנט הראשון שמועבר ל- [bind] {בדוגמטא שאנו מתארים: ערכו של sd הוא 3}
- ובכניסה זו שמור מצביע לרשומת סוקט. במקרה של הדוגמא שלנו מדובר ברשומת הסוקט של סוקט האזנה.
- TCPנ ייגש לרשומת סוקט זו, ומשנה את ערכו של השדה local port ברשומה, לערך שמועבר לו דרך הרשומה (ה- struct) ששמו srv
- srv הוא מטיפוס: struct sockaddr_in. וכולל שדה ששמו sin_port

- וברשומת הסוקט הזו, bind תרשום בשדה Local_Port את הערך 80 [= הערך שמועבר לה 'באמצעות' הארגומנט השני {= srv}]

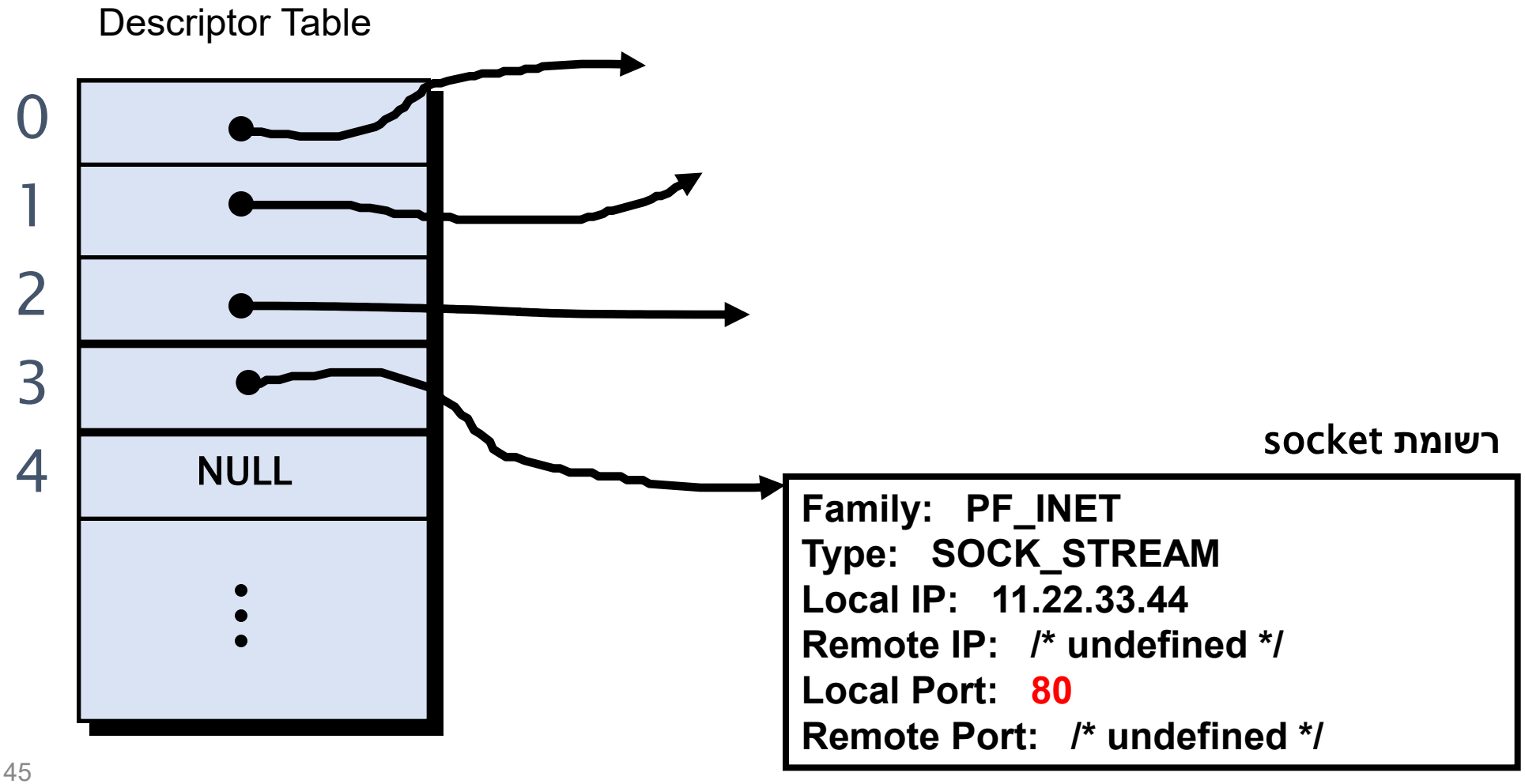
**The server's process socket sruct <u>before</u> invoking bind()**

בדוגמא הזו הקריאה
sd = socket(AF_INET, SOCK_STREAM, 0))
תחזיר ל- sd את הערך 3.



**The server's process socket sruct <u>after</u> invoking bind()**

הקריאה ל- bind(...) גורמת לעדכון ערך השדה Local Port ברשומת הסוקט מערך אקראי כלשהו שניתן עם הקצאת הסוקט, ל- 80.



*after invoking bind()*
*the server invokes listen()*

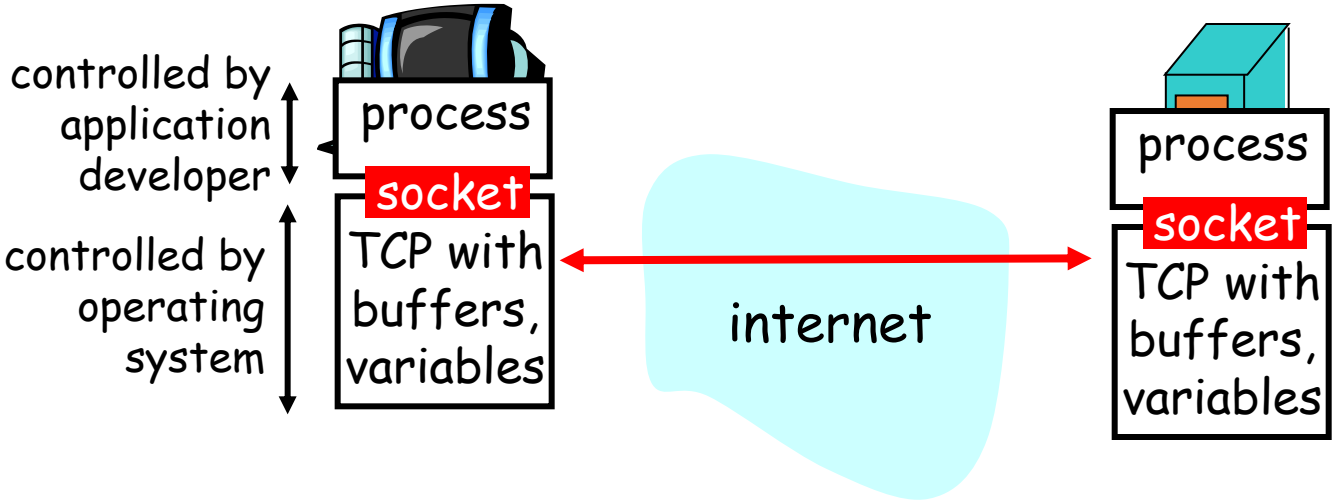
client program

running in a client computer C

a server program

running in a server computer S

socket( )
bind()
listen()



## How the server's process invokes listen() ?

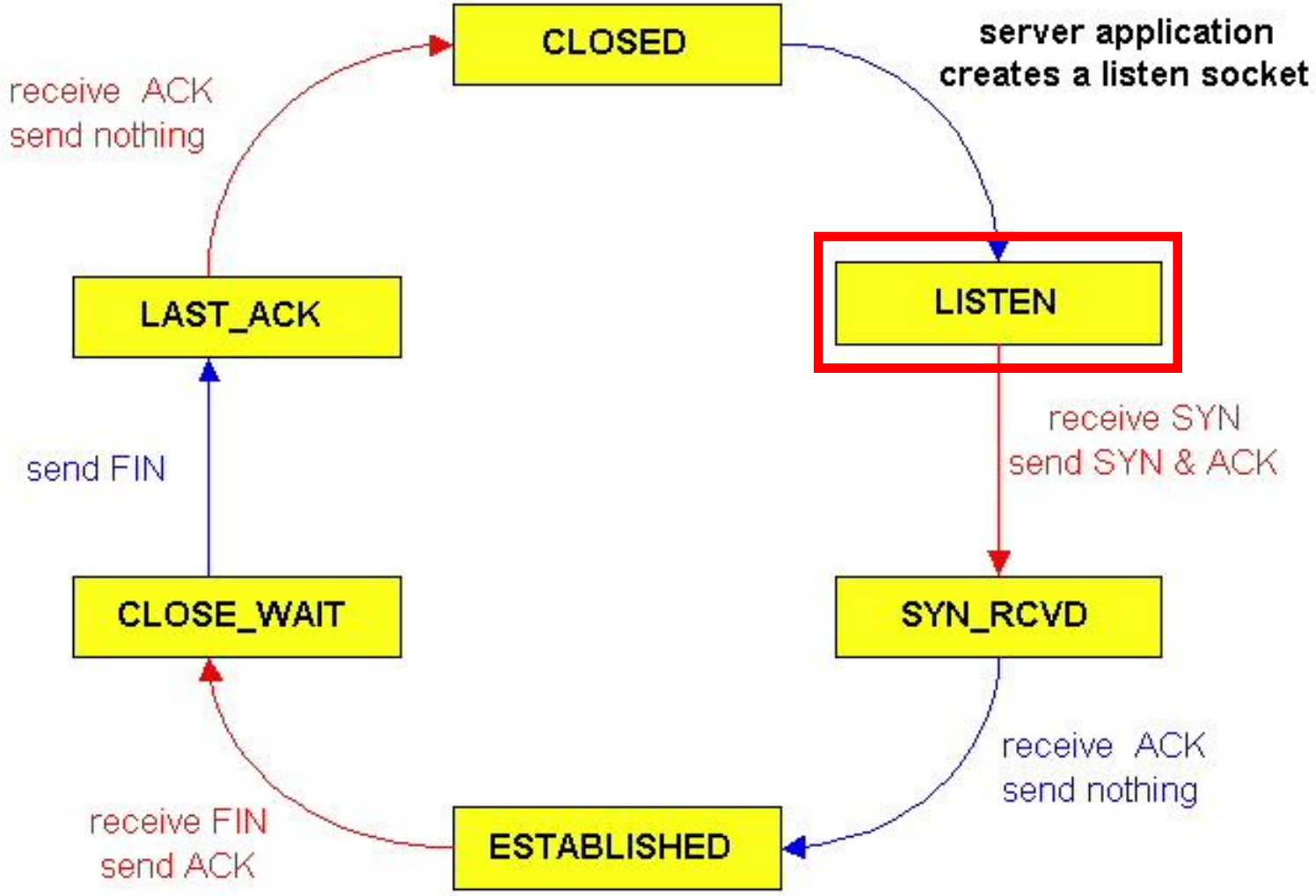
```
int sd;                                  /* socket descriptor */
struct sockaddr_in srv;  /* used by bind() */

/* 1)  create the socket */
/* 2)  bind the socket to a port */


/* 3) start listening for connections */
if(listen(sd, 5) < 0) {       /* backlog of 1 */
            perror("listen");
            exit(1);
}
```

## מה קורה בעקבות הקריאה ל- listen() ?

- בעקבות הקריאה ל- listen(), TCP במחשב השרת ישמור בתור (= לפי סדר הגעה) בקשות להקמת קישור [= בקשות connect] שיגיעו מתהליכי לקוח {שונים !}
- גודלו המקסימלי של תור זה מועבר כערך מסוג int בארגומנט השני של הקריאה ל- listen()

- בנוסף, הקריאה ל- listen() מעבירה את הסוקט למצב 'האזנה'.

**Server's Socket Lifecycle:**
**- after invoking listen()**



*So far, we described what are the steps carried out by the server's process in phase 0.*

*What about the client's process ?*
*The client's process must also invoke socket() !!*

*What happens when the client invokes socket() ?*

client program

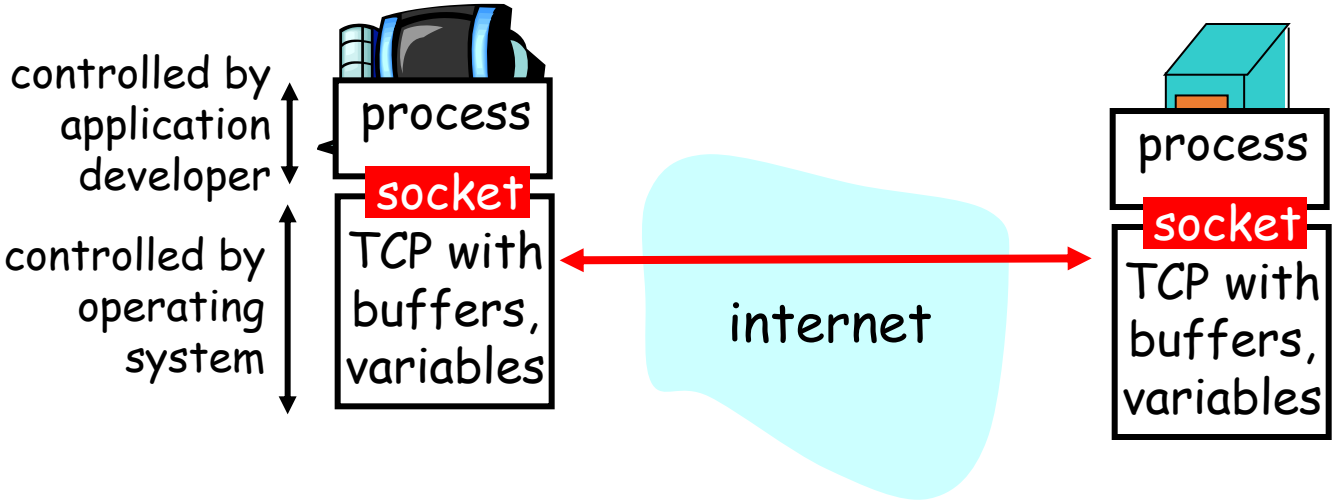
running in a client computer C

a server program

running in a server computer S

socket( )

socket( )
bind()
listen()

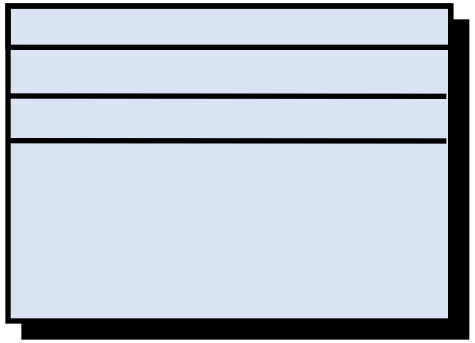


# Client Process's PCB

process_id:
parent_process_id:

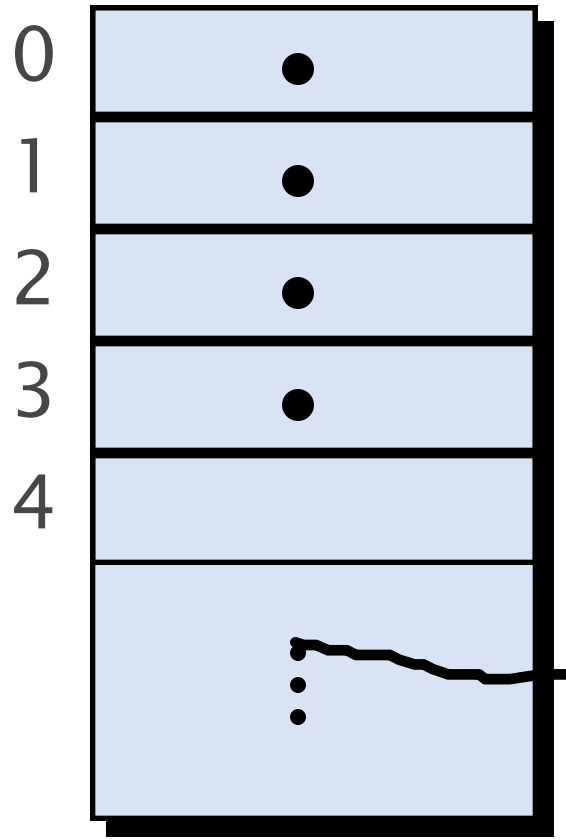
priority:
page table:

descriptor table:



מוקצה ומנוהל במע'
ההפעלה של מחשב C

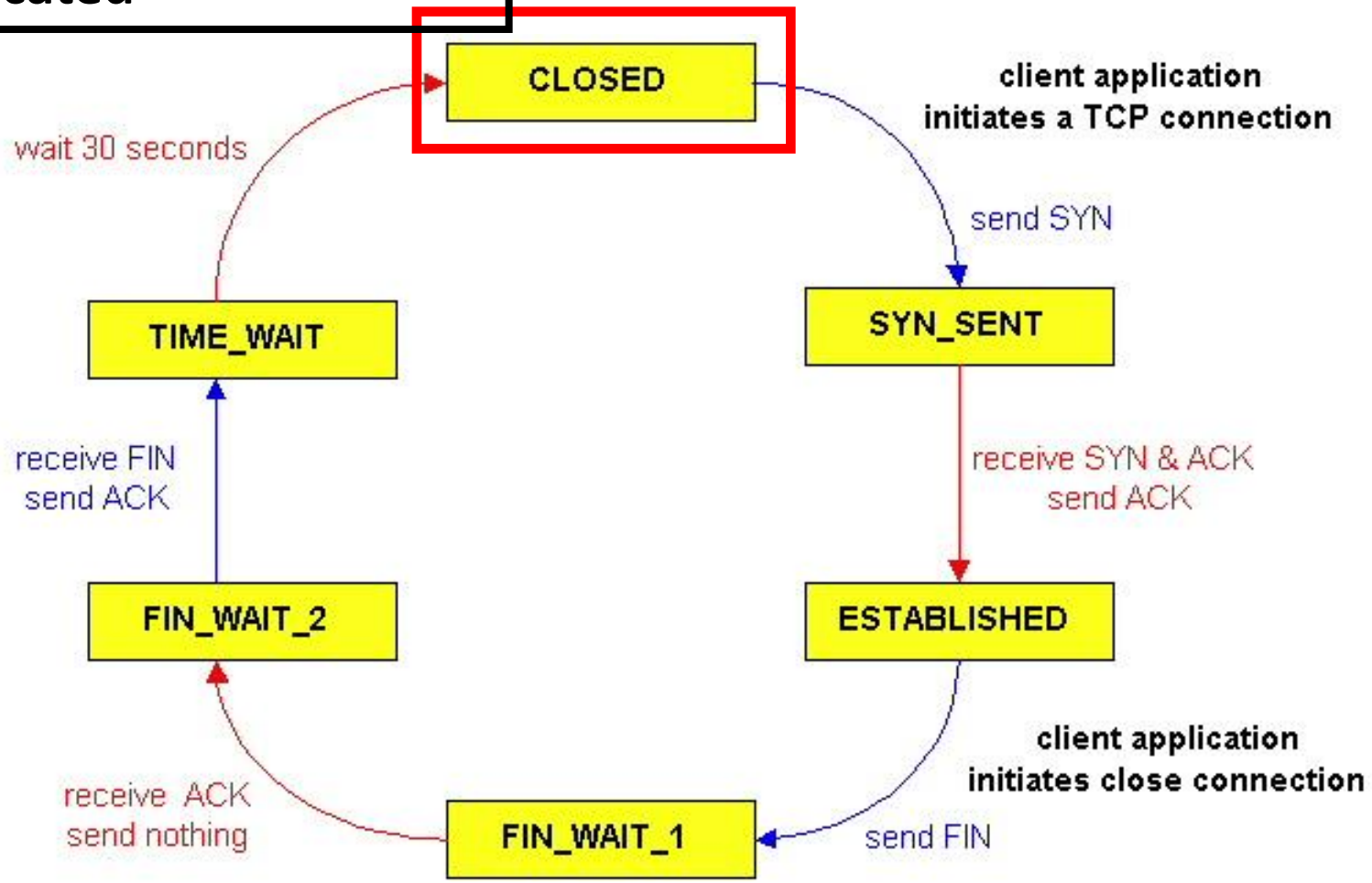
client's Process Descriptor Table




Local IP: 111.22.3.4
Remote IP: UNDEF
Local Port: 3456
Remote Port: UNDEF

- הערך בשדה Remote IP הוא undefined מכיוון שבשלב הזה לא הוקם עדיין קישור עם תהליך שרת.
- כנ"ל לגבי הערך בשדה Remote Port

**<u>Client's Socket Lifecycle</u>:**
**the client's socket state just after socket was allocated**



# *Phase 1*
# *TCP Connection Establishment: Handshake*

# TCP is connection oriented

- פרוטוקול TCP מאופיין כפרוטוקול **מוכְוָן קישור** [= connection oriented]
- היינו העברת הודעות באמצעות TCP כרוכה בשני שלבים:
  - שלב לחיצת היד
  - העברת הודעות דרך קישור שהוקם בשלב לחיצת היד

- היינו, לצורך שליחת הודעות בקשה, תהליך הלקוח חייב לקבל קודם לכן את 'הסכמתו' של תהליך השרת לכך שהוא ישלח לו הודעות בקשה.
- או: תהליך הלקוח:
  - חייב לשלוח בקשה להקמת קישור לתהליך השרת,
  - ולהמתין לאישור של תהליך השרת לבקשה.

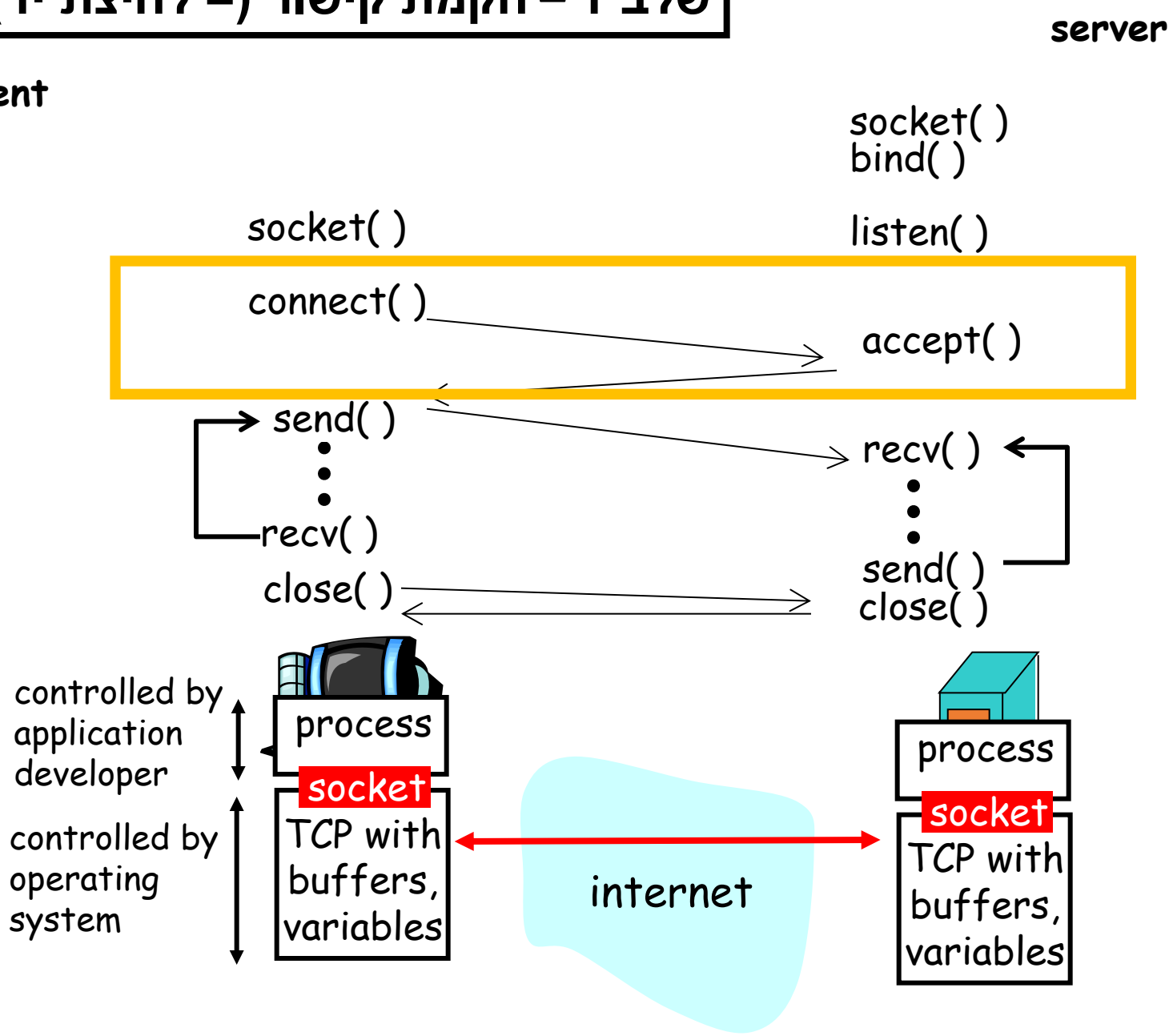
- שליחת הבקשה להקמת קישור (מצדו של תהליך הלקוח) + אישור הבקשה (מצדו של תהליך השרת) נקראים **לחיצת יד** [= handshake]

# לחיצת יד תלת-שלבית

*לחיצת היד המוגדרת ב- TCP היא **לחיצת יד תלת שלבית** [= **three-way handshaking**]*

*1. בשלב הראשון, תהליך הלקוח מבצע קריאה ל- connect()*
*2. בשלב השני, תהליך השרת מבצע קריאה ל- accept()*
*3. בצעד השלישי בלחיצת היד התלת-שלבית תהליך הלקוח מבצע קריאה ל- send() {או ל- write()}*

# שלב 1 – הקמת קישור (= לחיצת יד)



*The 1st step in the 3-way handshaking:*
*The client invokes connect() {or write()}*

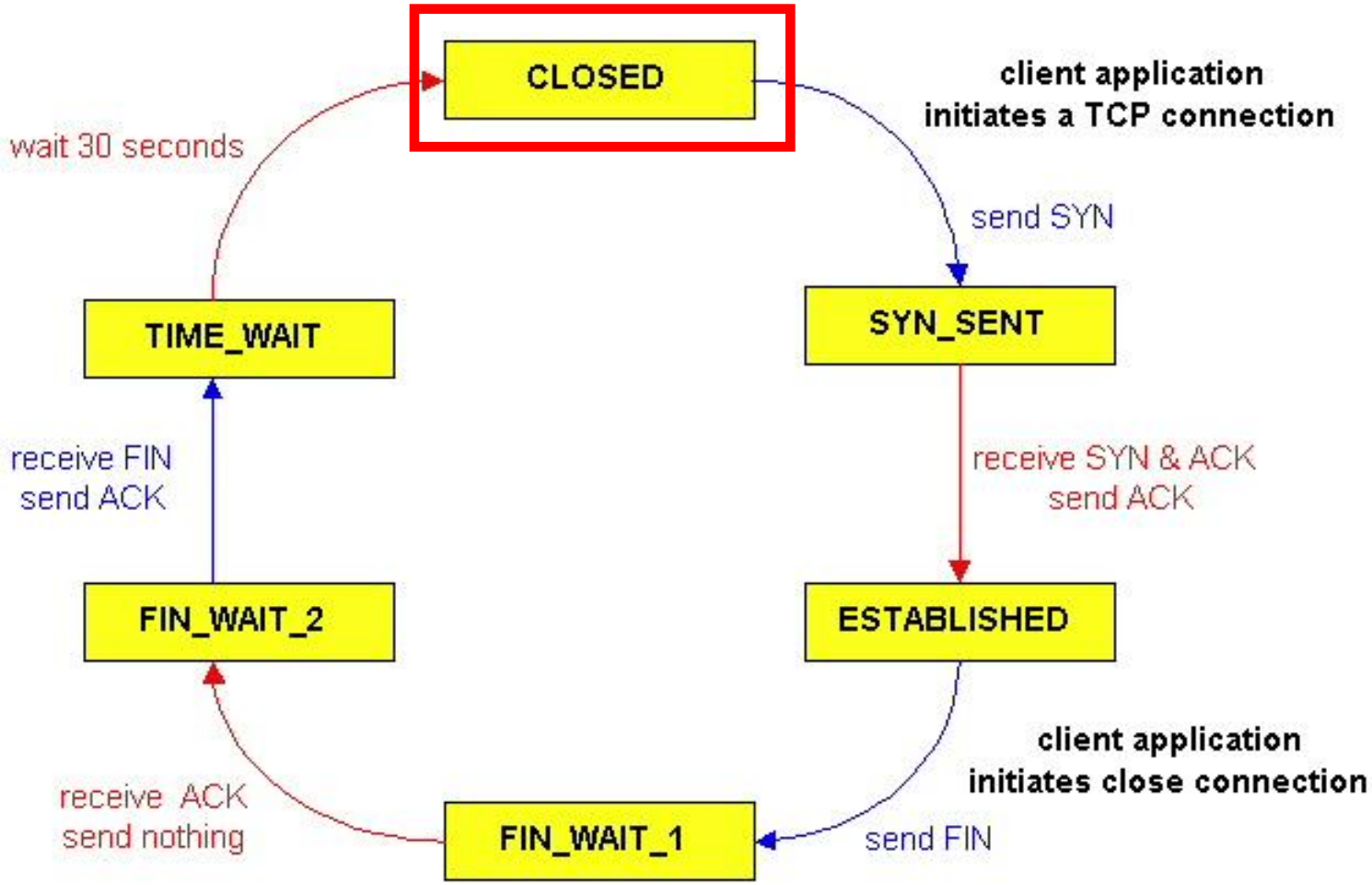
# connect()

```
int fd;                          /* socket descriptor */
struct sockaddr_in srv;          /* used by connect() */

/* create the socket */

/* connect: use the Internet address family */
srv.sin_family = AF_INET;

/* connect: socket ‘fd’ to port 80 */
srv.sin_port = htons(80);

/* connect: connect to IP Address “128.2.35.50” */
srv.sin_addr.s_addr = inet_addr(“128.2.35.50”);

if(connect(fd, (struct sockaddr*) &srv, sizeof(srv)) < 0) {
        perror(”connect"); exit(1);
}
```

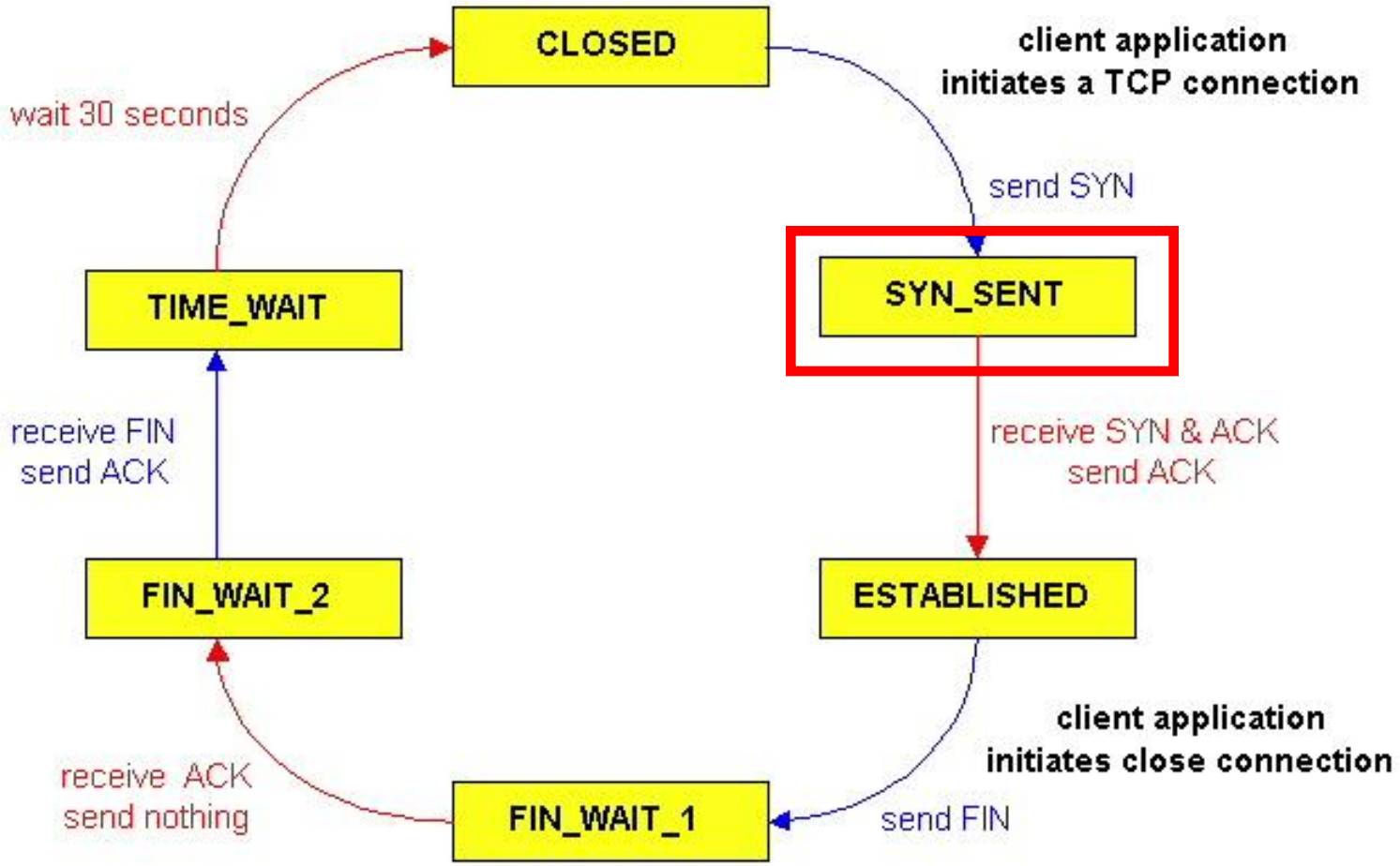
- connect יכולה כמובן להחזיר:
- ערך כישלון (= '1-').
  - במקרה שבקשת הקמת הקישור לא מאושרת
    - למשל, אם ה- P-server 'מתעלם' מאישור בקשת הקמת קישור
    - אחרי שעובר פרק זמן מסוים מרגע הקריאה ל- connect וה- P-server עדיין לא החזיר אישור (כי למשל התעלם מבקשת הקמת הקישור), אזי ה- connect יחזיר ערך '1-'
- ערך הצלחה (ערך 0).
  - ז"א בקשת הקמת הקישור אושרה על ידי ה- P-server
  - מה שאומר ש- P-client קיבל מה- P-server 'אור ירוק' לשלוח אליו הודעות בקשה (בקשה לשירות)

- P-server יכול שלא לאשר ל- P-client לשלוח לו הודעות ממס' סיבות:
  - המחשב S שבו רצה תוכנית השרת אינו online.
  - המחשב S הוא online אבל תכנית השרת אינה רצה, ואז כמובן גם שמודול ה- P-server אינו רץ.
  - המחשב S הוא online וגם תכנית השרת אינה רצה, אבל היא עמוסה מדי (= משרתת יותר מדי תוכניות לקוח בו זמנית).

# Client's Socket Lifecycle:
# <u>before</u> sending the SYN segment



# Client's Socket Lifecycle:
# <u>after</u> sending the SYN segment



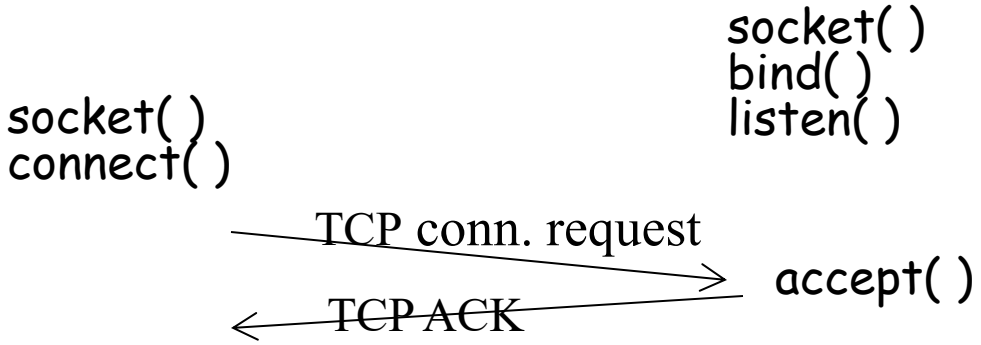
*The 2nd step in the 3-way handshaking:*
*The server invokes accept() {or write()}*

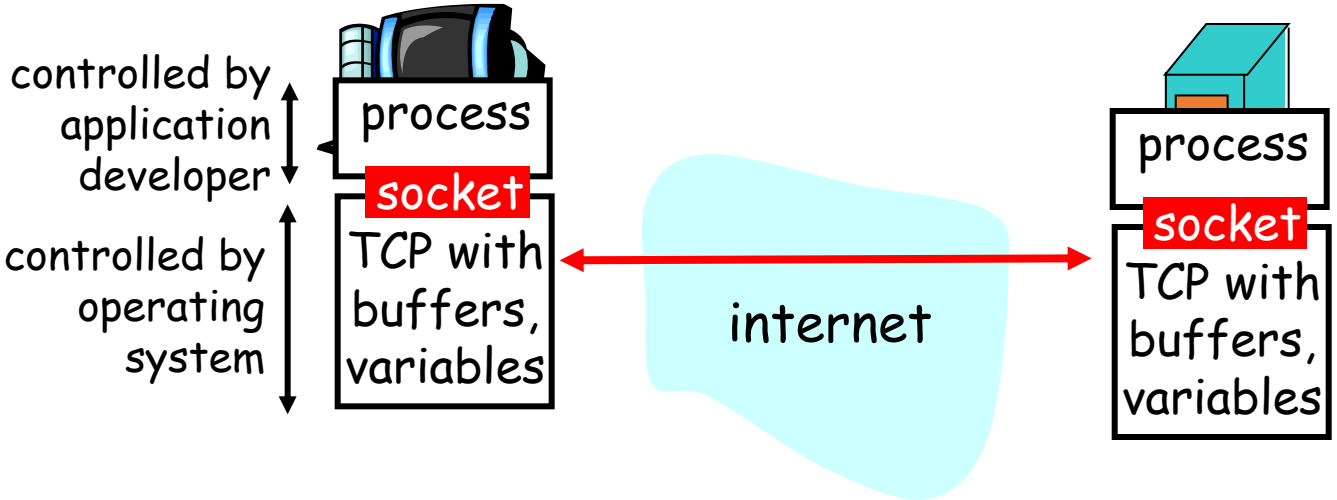
client program

running in a client computer C

a server program

running in a server computer S



# accept() – Receiving a client connection

**accept blocks waiting for a connection**

**accept returns a new socket (newsd) with the same properties as the original socket (sd)**

**newsd < 0 indicates that an error occurred**

```
int sd;                         /* socket descriptor */
struct sockaddr_in srv;         /* used by bind() */
struct sockaddr_in cli;         /* used by accept() */
int newsd;                      /* returned by accept() */
int cli_len = sizeof(cli);      /* used by accept() */

/* 1) create the socket */
/* 2) bind the socket to a port */
/* 3) listen on the socket */

newsd = accept(sd, (struct sockaddr*) &cli, &cli_len);
if(newsd < 0) {
        perror("accept");
        exit(1);
}
```

# accept() – Receiving a client connection (continued)

```
struct sockaddr_in cli;             /* used by accept() */
int newsd;                          /* returned by accept() */
int cli_len = sizeof(cli);   /* used by accept() */

newsd = accept(sd, (struct sockaddr*) &cli, &cli_len);
if(newsd < 0) {
        perror("accept");
        exit(1);
}
```

**How does the server know which client it is?**

**cli.sin_addr.s_addr contains the client's IP address**

**cli.sin_port contains the client's port number**

**Now the server can exchange data with the client by using read and write on the descriptor newsd**

# *Setting the sockaddr_in struct*

- בקשת הקמת הקישור, בקשת ה- connect, מופנית כמובן לתהליך שרת מסוים שרץ במחשב שרת מסוים.
- ולכן לצורך שליחת בקשת ה- connect על תהליך הלקוח לציין
  - את כת' ה- IP של מחשב השרת
  - את מס' הפורט של תהליך השרת (של P-server)

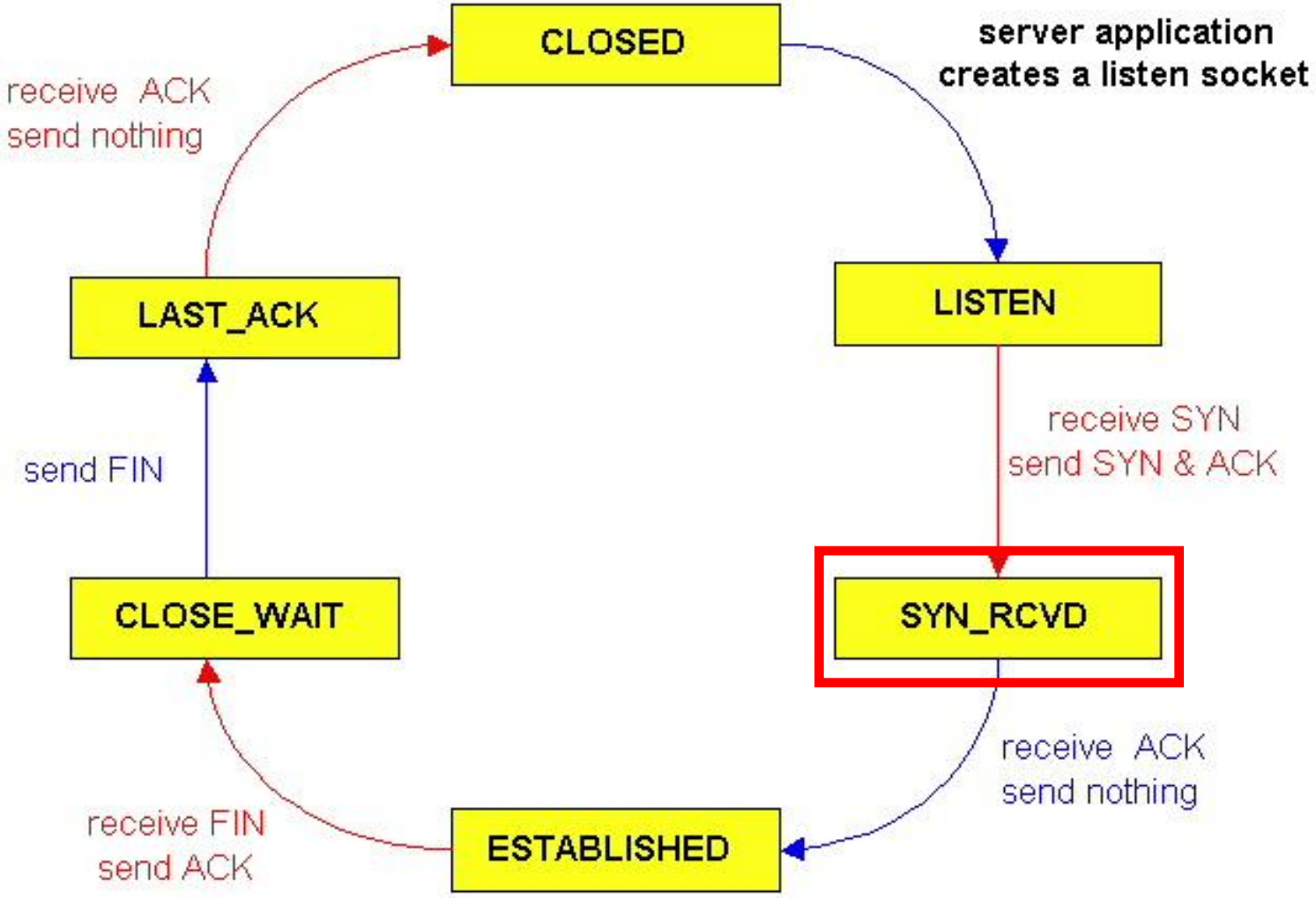
- את שני הערכים האלו ה- P-client רושם (או: שומר) ברשומה שמיועדת לכך
  - רשומה מסוג sockaddr_in
- באחד השדות של הרשומה ה- P-client רושם את כת' ה- IP של מחשב השרת
- ובשדה אחר, את מס' הפורט של P-server

- ומעביר את הרשומה כפרמטר בקריאה ל- ()connect

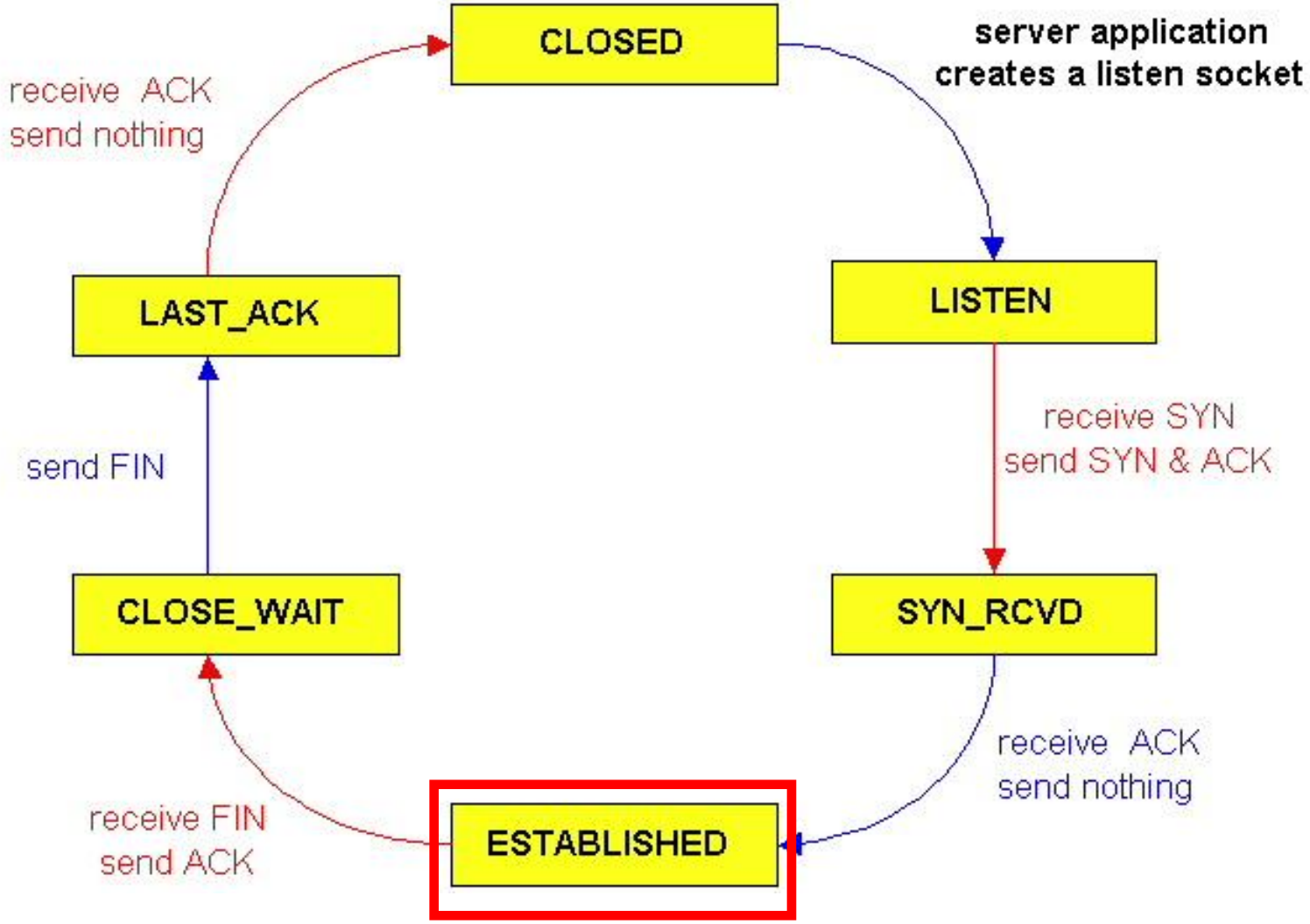
## Set up the sockaddr_in structure

```
struct  sockaddr_in    srv;
int port = 80;

memset(&srv, 0, sizeof(srv)); /* clear it */
srv.sin_family = AF_INET; /* network socket */

/* accept connections on any interface */
srv.sin_addr.s_addr = htonl(INADDR_ANY);

/* listen on the given port */
srv.sin_port = htons(port);
```

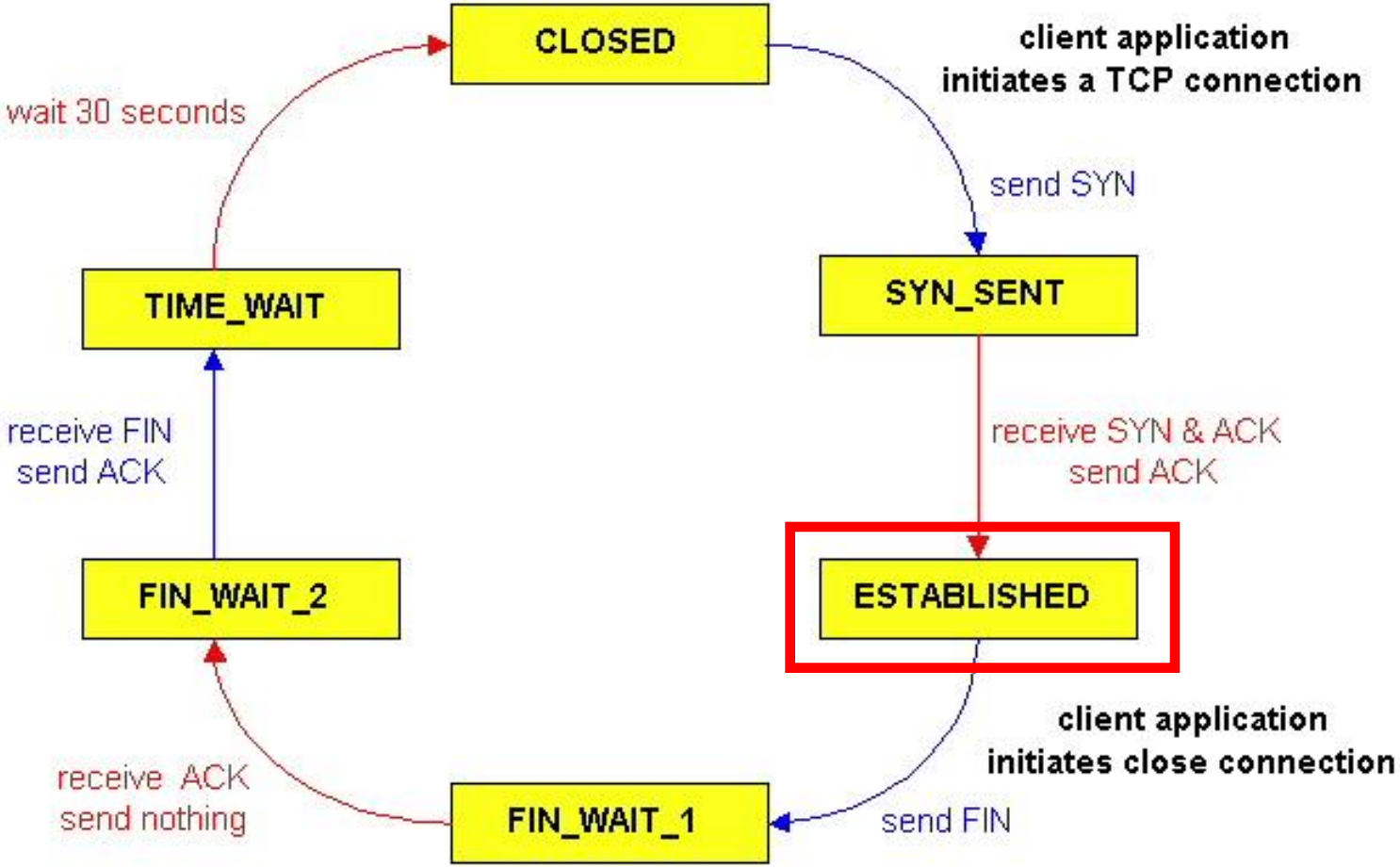
**Server's Socket Lifecycle:**
- just after receiving the SYN segment
- and before sending the SYN-ACK segment [= before invoking accept() ]



**Server's Socket Lifecycle:**
**- after sending the SYN-ACK segment [= before invoking accept() ]**

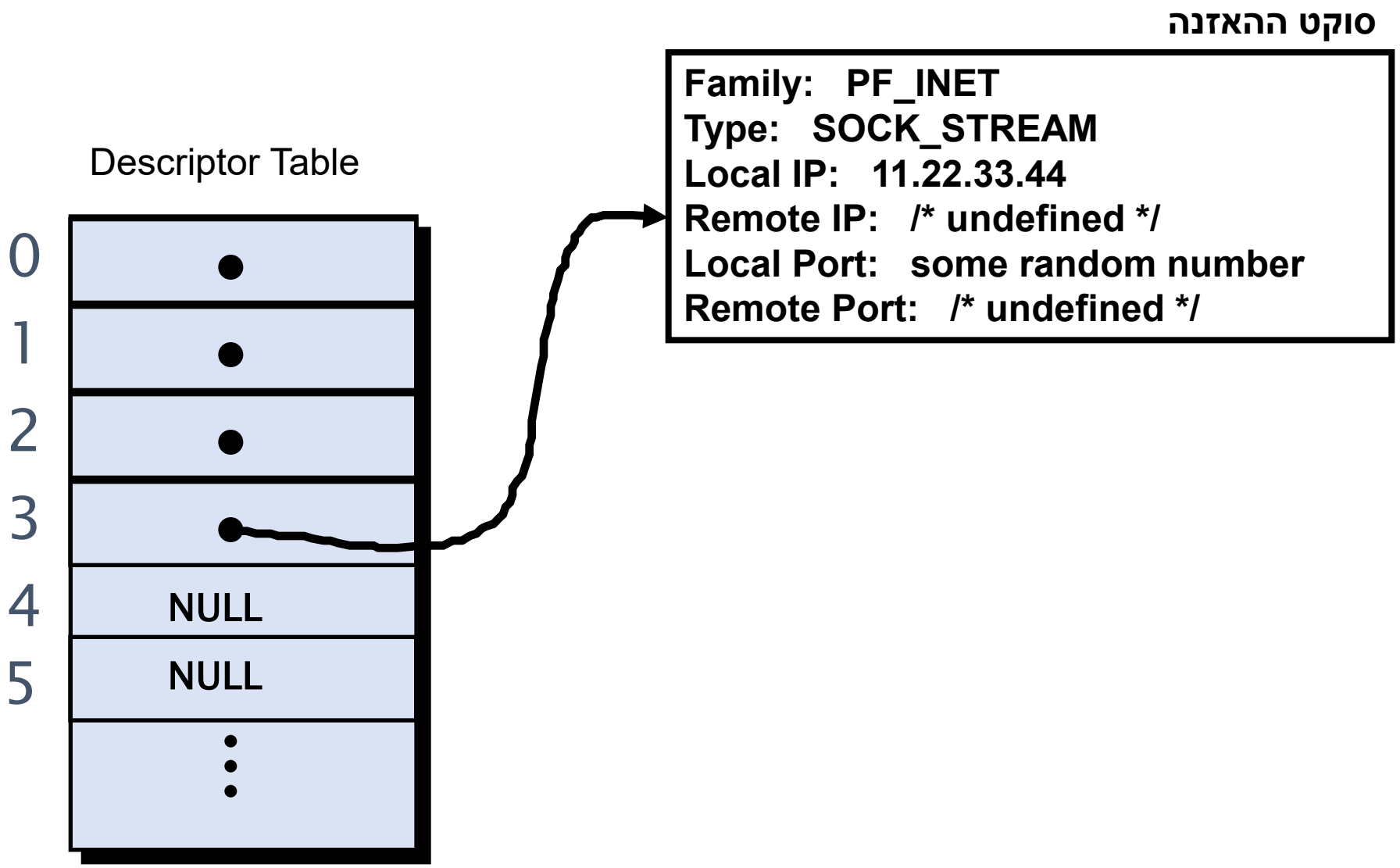


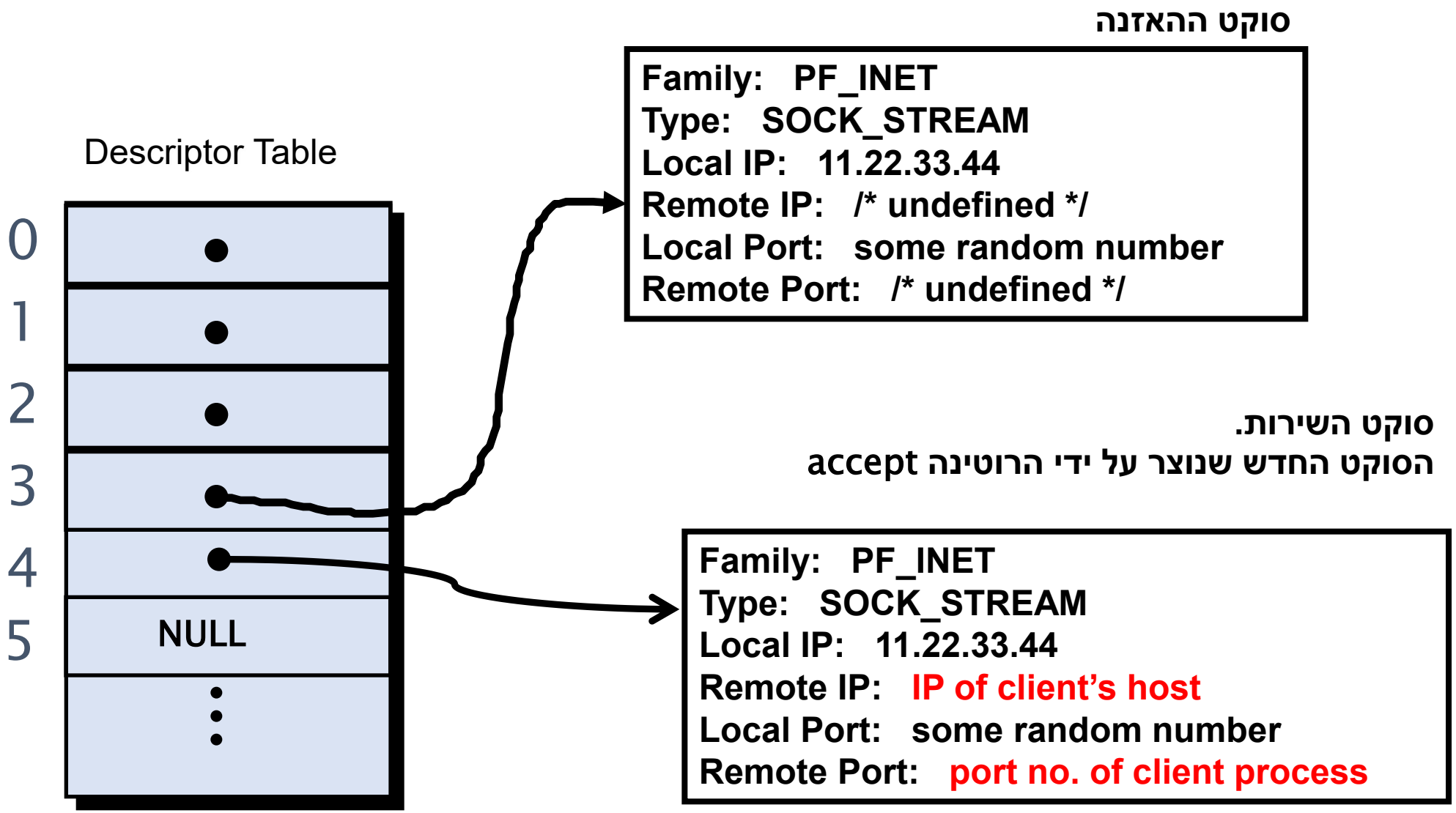
**<u>Client's Socket Lifecycle</u>:**
**- <u>after</u> receiving the SYN-ACK segment [= after connect() returns ]**



# *accept - implementation*

- ה- SYN segment נשלח כאמור מ- TCP במחשב הלקוח ל- TCP במחשב השרת.
- כאשר הוא מגיע ל- TCP במחשב השרת, הדבר גורם ל:
- תהליך התהליך השרת 'יוצא' ממצב המתנה אליו הוא נכנס בעקבות הקריאה שלו ל- accept()
- לשליחת SYN-ACK segment מ- TCP במחשב השרת ל- TCP במחשב הלקוח
- הקצאה של סוקט 'חדש' שישמש כסוקט שירות
- הסוקט 'המקורי' (זה שנוצר עם תחילת הריצה של תהליך השרת) ימשיך לשמש ל'האזנה' לבקשות connect שתגענה מתהליכי לקוח. וסוקט ה'שירות' ישמש לצורך החלפת הודעות בקשה ותשובה בין תהליך הלקוח שביצע את ה- connect ובין תהליך השרת.


סוקט ההאזנה
Family: PF_INET
Type: SOCK_STREAM
Local IP: 11.22.33.44
Remote IP: /* undefined */
Local Port: some random number
Remote Port: /* undefined */
Descriptor Table
0
1
2
3
4
5
NULL
NULL


סוקט ההאזנה
Family: PF_INET
Type: SOCK_STREAM
Local IP: 11.22.33.44
Remote IP: /* undefined */
Local Port: some random number
Remote Port: /* undefined */
Descriptor Table
0
1
2
3
4
5
NULL
סוקט השירות.
הסוקט החדש שנוצר על ידי הרוטינה accept
Family: PF_INET
Type: SOCK_STREAM
Local IP: 11.22.33.44
Remote IP: IP of client's host
Local Port: some random number
Remote Port: port no. of client process

- המקור ישמש לקבלת פניות להקמת קישור מ- clients
- המשוכפל ישרת את ה- client שזה עתה פנה, ובקשתו להקמת קישור אושרה.
- המקור יבצע (שוב) listen על סוקט ההאזנה. כדי לקבל פניות להקמת קישור מצד clients אחרים.
- המשוכפל יבצע recv מסוקט השירות לצורך קבלת הודעות בקשה מה- client שזה עתה פנה.

*UDP connection*

# קישור UDP

- שלבים עיקריים:
  - שלב של: Data exchange (החלפת הודעות בקשת שירות והודעות תשובה)
  - ואחריו שלב: Termination (סגירת הקישור)

- במהלך שלב 2 **<u>לא</u>** ממומשים מנגנוני:
  - ה- RDT
  - ה- flow control
  - וה- congestion control

# UDP client

- cli_sd = socket(…., SOCK_DGRAM, …) ;
- if( cli_sd == - 1) return(ERROR_CODE) ;
- **/* can send requests and receive responses */**

# UDP Server

- srv_sd = socket(…., SOCK_DGRAM, …) ;
- if( cli_sd == - 1) return(ERROR_CODE) ;

- bind(srv_sd,  /* the port no. I want to have */) ;
- /*  NO listen()  !!!! */